



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA RAČUNARSTVO I INFORMATIKU

Eksperimentalna analiza timing side-channel napada na RSA

ZAVRŠNI RAD
- PRVI CIKLUS STUDIJA -

Student:
Tarik Fetahović

Mentor:
Prof. dr Saša Mrdović.

Sarajevo,
maj 2026.

Sažetak

U radu se eksperimentalno analiziraju timing i cache side-channel napadi na RSA kriptosistem. Fokus je na statističkoj detekciji i iskorištavanju vremenskih razlika uzrokovanih uslovnim grananjem u square-and-multiply algoritmu, te na Flush+Reload napadu koji iskorištava dijeljeni LLC (*Last Level Cache*). Provedeno je sedam eksperimenata na modernom procesoru (Intel i5-1335U): detekcija timing leaka, averaging napad, potpuna rekonstrukcija 64-bitnog privatnog ključa, sistematska multi-bit analiza, evaluacija utjecaja kompajlerskih optimizacija, analiza RSA blindinga kao kontramjere, te Flush+Reload cache napad.

Rezultati pokazuju da naivna implementacija ima mjerljiv timing leak od ≈ 76 CPU ciklusa po bitu ($p < 10^{-50}$, Cohen-ov $d = 0,134$), koji omogućava potpunu slijepu rekonstrukciju ključa sa 100% tačnošću uz 5000 mjerenja po bitu (teorijski dovoljno i $K \approx 2000$ za 100% tačnost po bitu). Constant-time implementacija potpuno eliminiše leak ($p = 0,84$, $d \approx 0$). Kompajlerske optimizacije (O0–O3) ne eliminišu ranjivost, a RSA blinding ne štiti od branch-zavisnih leaka. Flush+Reload napad potvrđuje bimodalnu distribuciju reload latencija s razlikom od ≈ 342 ciklusa između cache hit i cache miss, što omogućava klasifikaciju bita iz jednog mjerenja.

Ključne riječi: RSA, timing side-channel napad, Flush+Reload, square-and-multiply, constant-time, statistička analiza, rekonstrukcija ključa, RSA blinding

Abstract

This thesis experimentally analyzes timing and cache side-channel attacks on the RSA cryptosystem, focusing on statistical detection and exploitation of timing differences caused by conditional branching in the square-and-multiply algorithm, as well as the Flush+Reload attack exploiting the shared Last Level Cache (LLC). Seven experiments were conducted on a modern processor (Intel i5-1335U): timing leak detection, averaging attack, full 64-bit private key reconstruction, systematic multi-bit analysis, compiler optimization impact evaluation, RSA blinding countermeasure analysis, and Flush+Reload cache attack.

Results demonstrate that the naive implementation exhibits a measurable timing leak of ≈ 76 CPU cycles per bit ($p < 10^{-50}$, Cohen's $d = 0.134$), enabling complete blind key reconstruction with 100% accuracy using 5000 measurements per bit (with $K \approx 2000$ theoretically sufficient for 100% per-bit accuracy). The constant-time implementation fully eliminates the leak ($p = 0.84$, $d \approx 0$). Compiler optimizations (O0–O3) do not eliminate the vulnerability, and RSA blinding does not protect against branch-dependent leaks. The Flush+Reload attack confirms a bimodal reload latency distribution with ≈ 342 cycle separation between cache hit and miss, enabling single-measurement bit classification.

Keywords: RSA, timing side-channel attack, Flush+Reload, square-and-multiply, constant-time, statistical analysis, key reconstruction, RSA blinding

Postavka zadatka završnog rada I ciklusa: Eksperimentalna analiza timing side-channel napada na RSA

U okviru rada je potrebno razviti $\text{\LaTeX}$ predložak za izradu završnog rada prvog ciklusa studija. U radu je potrebno:

- objasniti opći postupak za izradu i pisanje završnih radova prvog ciklusa studija, poštujući odgovarajuće Pravilnike,
- dati kratko uputstvo za korištenje .tex dokumenata, pisanje slika, tabela i relacija, generisanje sadržaja, popisa slika i tabela i sl.
- dati kratko uputstvo za generiranje odgovarajućih .pdf dokumenata iz odgovarajućih .tex fajlova,

Preporučuje se korištenje TexLive $\text{\LaTeX}$ podrške, verzije 2013 ili novije.

Napomena: U dijelu "Postavka" se stavlja postavka rada koju je specificirao mentor prilikom davanja teme. Naročito obratiti pažnju na naslov rada koji mora biti konzistentan tokom cijelog dokumenta, i usaglašen sa temom upisanom u informacioni sistem (npr. Zamger). Na ovoj stranici se potpisuje mentor prije nego se radovi odnesu u studentsku službu.

Polazna literatura:

- [1] Sonka, M., Hlavac, V., Boyle, R., "Image Processing, Analysis, and Machine Vision", Thomson, 2008.
- [2] Lee, E.A., Varaiya, P., "Structure and Interpretation of Signals and Systems", Electrical Engineering & Computer Science University of California, Berkeley, 2000

Izjava o autentičnosti radova

Završni rad I ciklusa studija

Ime i prezime: Tarik Fetahović

Naslov rada: Eksperimentalna analiza timing side-channel napada na RSA

Vrsta rada: Završni rad Prvog ciklusa studija

Broj stranica:

Potvrđujem:

- da sam pročitao dokumente koji se odnose na plagijarizam, kako je to definirano Statutom Univerziteta u Sarajevu, Etičkim kodeksom Univerziteta u Sarajevu i pravilima studiranja koja se odnose na I i II ciklus studija, integrirani studijski program I i II ciklusa i III ciklus studija na Univerzitetu u Sarajevu, kao i uputama o plagijarizmu navedenim na web stranici Univerziteta u Sarajevu;
- da sam svjestan univerzitetskih disciplinskih pravila koja se tiču plagijarizma;
- da je rad koji predajem potpuno moj, samostalni rad, osim u dijelovima gdje je to naznačeno;
- da rad nije predat, u cjelini ili djelimično, za stjecanje zvanja na Univerzitetu u Sarajevu ili nekoj drugoj visokoškolskoj ustanovi;
- da sam jasno naznačio prisustvo citiranog ili parafraziranog materijala i da sam se referirao na sve izvore;
- da sam dosljedno naveo korištene i citirane izvore ili bibliografiju po nekom od preporučenih stilova citiranja, sa navođenjem potpune reference koja obuhvata potpuni bibliografski opis korištenog i citiranog izvora;
- da sam odgovarajuće naznačio svaku pomoć koju sam dobio pored pomoći mentora i akademskih tutora/ica.

Sarajevo, maj 2026.

Potpis:

(Tarik Fetahović)

Sadržaj

Popis slika	vii
Popis tabela	viii
1 Uvod	1
1.1 Obrazloženje teme	1
1.1.1 Doprinos rada	2
1.2 Struktura rada	2
2 Teorijske osnove	3
2.1 RSA kriptosistem	3
2.1.1 Generisanje ključeva	3
2.1.2 Šifriranje i dešifriranje	3
2.1.3 Sigurnost RSA-a	4
2.2 Modularna eksponencijacija	4
2.2.1 Kvadriraj-i-množi algoritam	4
2.2.2 Vremenska ranjivost algoritma	5
2.3 Side-channel napadi	6
2.3.1 Definicija i klasifikacija	6
2.3.2 Kocher-ov timing napad (1996)	6
2.3.3 Brumley-Boneh mrežni napad (2003)	6
2.3.4 Cache timing side-channel napadi	7
2.3.5 Kontramjera: Constant-time implementacija	8
2.3.6 Kontramjera: RSA blinding	8
2.3.7 Model u ovom radu	8
2.4 Statističke metode u analizi side-channel napada	9
2.4.1 Problem signala i šuma	9
2.4.2 Usrednjavanje i centralna granična teorema	10
2.4.3 Welch-ov t-test	10
2.4.4 Mann-Whitney U test	11
2.4.5 Cohen-ova mjera veličine efekta	11
2.4.6 Upareni t-test	11
2.4.7 Bootstrap interval povjerenja	12
2.4.8 Power analiza	12
2.5 Hardverske karakteristike relevantne za eksperiment	12
2.5.1 Time Stamp Counter (TSC)	12
2.5.2 Izvori šuma u mjerenjima	12

3	Implementacija i eksperimentalni dizajn	13
3.1	Formalni eksperimentalni okvir	13
3.1.1	Hipoteze	13
3.1.2	Eksperimentalne varijable	13
3.2	Testna platforma i priprema okruženja	13
3.2.1	Specifikacije testne platforme	13
3.2.2	Kontrola eksperimentalnih uslova	14
3.3	Implementacija mjerenja vremena	14
3.3.1	RDTSC s CPUID serijalizacijom	14
3.3.2	Ograničenja kompajlera	15
3.4	Implementacija RSA modularne eksponencijacije	15
3.4.1	Modularna multiplikacija	15
3.4.2	Ranjiva implementacija	16
3.4.3	Constant-time implementacija	16
3.5	Dizajn eksperimenta prikupljanja podataka	17
3.5.1	Dizajn uparenih mjerenja	17
3.5.2	Nasumičan redoslijed	17
3.5.3	Filtriranje rubnih vrijednosti	18
3.5.4	Parametri eksperimenta	18
3.6	Implementacija Flush+Reload napada	18
3.6.1	Dijeljeni RSA kod	19
3.6.2	Žrtva i napadač	19
3.6.3	Kalibracija praga	20
3.7	Napad rekonstrukcije ključa	20
3.7.1	Struktura napada i model slijepe rekonstrukcije	20
3.7.2	Princip bit-po-bit napada	21
3.7.3	Tehnike robustnosti	21
4	Rezultati i analiza	22
4.1	Eksperiment 1: Detekcija timing leaka	22
4.1.1	Deskriptivna statistika	22
4.1.2	Statistički testovi	23
4.1.3	Vizualizacija distribucija	24
4.1.4	Validacija eksperimenta	25
4.2	Eksperiment 2: Napad sa usrednjavanjem	25
4.2.1	Metodologija	25
4.2.2	Rezultati	26
4.2.3	Osjetljivost na veličinu uzorka	27
4.3	Eksperiment 3: Slijepa rekonstrukcija 64-bitnog privatnog ključa	28
4.3.1	Metodologija	28
4.3.2	Rezultati	28
4.4	Eksperiment 4: Analiza pozicije bita	30
4.4.1	Motivacija i metodologija	30
4.4.2	Rezultati	30
4.5	Eksperiment 5: Utjecaj kompajlerskih optimizacija	32
4.5.1	Motivacija	32
4.5.2	Rezultati	32
4.6	Eksperiment 6: Evaluacija RSA blindinga	33

4.6.1	Motivacija	33
4.6.2	Rezultati	33
4.6.3	Analiza: zašto blinding ne eliminiše leak	34
4.7	Eksperiment 7: Flush+Reload cache napad	35
4.7.1	Motivacija	35
4.7.2	Princip Flush+Reload napada	35
4.7.3	Implementacija	35
4.7.4	Rezultati	36
4.7.5	Poređenje s timing napadom	36
4.8	Diskusija	37
4.8.1	Interpretacija Cohen-ovog $d = 0,134$	37
4.8.2	Praktična cijena napada na realni RSA	37
4.8.3	Primjenjivost na moderne RSA biblioteke	38
5	Zaključak	39
5.1	Pregled postignutih rezultata	39
5.2	Implikacije za praktičnu kriptografsku sigurnost	39
5.2.1	Neintuitivnost ranjivosti	39
5.2.2	Zavisnost od kompajlera nije sigurnost	40
5.2.3	Blinding ima ograničenja	40
5.3	Ograničenja i mogući pravci daljeg istraživanja	40
5.3.1	Ograničenja	40
5.3.2	Pravci daljeg istraživanja	40
	Prilozi	42
A	Konfiguracija hardverskog okruženja	43
A.1	Isključivanje Turbo Boost-a	43
A.2	Performance CPU governor	43
A.3	CPU pinning	43
A.4	Zagrijavanje cache-a	44
A.5	Verifikacija TSC frekvencije	44
A.6	Provjera assembly izlaza	44
A.7	Assembly izlaz: implementacija sa grananjem i bez grananja	44
A.7.1	Ranjiva implementacija (eksplicitni branch)	44
A.7.2	Konstantna vremenska implementacija (branchless)	45
	Literatura	46

Popis slika

2.1	Dijagram toka kvadriraj-i-množi algoritma.	5
2.2	Faze Flush+Reload napada	7
4.1	Distribucija trajanja ranjive implementacije: histogram, boxplot i CDF za grupe bit=0 i bit=1.	24
4.2	Distribucija parnih razlika trajanja za ranjivu (crvena) i konstantnu vremensku (zelena) implementaciju.	25
4.3	Dijagram raspršenja (<i>scatter plot</i>) mjerenja (lijevo) i tačnost u funkciji K (desno).	26
4.4	Osjetljivost napada na veličinu uzorka: tačnost u funkciji N za $K = 100$	28
4.5	Gornji panel: timing razlika Δ za svaki od 64 bita (pozitivna vrijednost = bit=1, negativna = bit=0). Srednji panel: toplotna mapa rekonstruisanog i stvarnog ključa. Donji panel: distribucija svih Δ vrijednosti	29
4.6	Timing razlika Δ po poziciji bita eksponenta sa 95% intervalima povjerenja.	31
4.7	Utjecaj kompajlerskih optimizacija: prosječna timing razlika Δ (u CPU ciklusima) i Cohen-ov d za pet varijanti ranjive implementacije.	32
4.8	Evaluacija RSA blindinga kao kontramjere: distribucija parnih timing razlika za blinded (zelena) i unblinded (crvena) varijantu	34
4.9	Histogram reload latencija: bimodalna distribucija, vremenska sekvenca mjerenja i rolling hit rate	36

Popis tabela

3.1	Specifikacije testne platforme	14
3.2	Parametri eksperimenta	18
4.1	Deskriptivna statistika trajanja modularne eksponencijacije (u CPU ciklusima) .	22
4.2	Rezultati statističkih testova	23
4.3	Validacija eksperimenta: svi kriteriji ispunjeni	25
4.4	Tačnost timing napada sa usrednjavanjem u funkciji broja usrednjenih mjerenja K	26
4.5	Osjetljivost na veličinu uzorka: p-vrijednost i tačnost napada	27
4.6	Rezultati slijepe rekonstrukcije 64-bitnog ključa	28
4.7	Multi-bit sweep: ukupni rezultati	30
4.8	Timing leak po nivoima kompajlerske optimizacije	32
4.9	Evaluacija RSA blindinga kao kontramjere	33
4.10	Rezultati Flush+Reload napada na <code>rsa_multiply_step()</code>	36
4.11	Poređenje timing napada i Flush+Reload napada	37

Poglavlje 1

Uvod

1.1 Obrazloženje teme

Sigurnosni protokoli poput HTTPS-a ili SSH-a u pozadini koriste asimetrične kriptografske algoritme da uspostave sigurnu vezu, a RSA je među njima jedan od najrasprostranjenijih. Sigurnost RSA-a se zasniva na težini faktORIZACIJE proizvoda dva vrlo velika prosta broja [1].

Matematička robusnost kriptografskog algoritma ne implicira i sigurnost njegove konkretne implementacije. Softver može nenamjerno otkriti informacije o tajnom ključu kroz mjerljive fizičke popratne efekte kao što je vrijeme izvršavanja koda ili varijacije u naponu uređaja na kojem je pokrenut algoritam. Takvi napadi se klasifikuju kao napadi bočnim kanalima (*side-channel attacks*), pri čemu se umjesto matematičkih slabosti napada fizički proces izvođenja operacija.

Standardna implementacija RSA modularne eksponencijacije zasniva se na algoritmu square-and-multiply. U njemu se za svaki bit privatnog eksponenta čija je vrijednost 1 izvodi dodatna operacija množenja koja nije prisutna kada je vrijednost bita 0. Budući da to jedno množenje unosi mjerljivo vremensko odstupanje od nekoliko desetina CPU ciklusa, napadač koji može mjeriti razlike u trajanju operacija dešifriranja dovoljan broj puta može statističkom metodom usrednjavanja, bit po bit, rekonstruisati tajni ključ. Ovo je Paul Kocher demonstrirao 1996. godine na stvarnim uređajima [2], a Jean-François Dhem i koautori su pokazali konkretnu praktičnu realizaciju napada [3]. Nekoliko godina kasnije, Brumley i Boneh su pokazali da napad funkcioniše i kroz mrežu, na različitim platformama i implementacijama, ciljajući stvarni OpenSSL server [4].

RSA je odabran kao cilj analize jer je najrasprostranjeniji asimetrični algoritam u postojećim sistemima, a square-and-multiply algoritam pruža jasan i izolovan timing signal pogodan za eksperimentalnu demonstraciju.

Za razliku od Brumley i Boneh koji su analizirali mrežni OpenSSL server, ovaj rad demonstrira timing napad u lokalnom okruženju na modernom procesoru s modernijom arhitekturom i provodi potpunu rekonstrukciju 64-bitnog privatnog ključa isključivo iz timing mjerenja.

Suštinski istraživački problem ovog rada je detekcija slabog signala u okruženju sa visokim nivoom šuma. Vremenska razlika koju unosi jedna operacija množenja zanemarljiva je u poređenju sa sistemskim šumom koji proizvode moderni procesori i operativni sistemi. Međutim, primjenom statističkih metoda usrednjavanja šum konvergira prema nuli, što omogućuje ekstrakciju tajnog ključa iz dovoljno velike serije uzoraka. Kako se ističe u sveobuhvatnom pregledu mikroarhitekturnih timing napada [5], upravo ova mogućnost ekstrakcije ključa iz naivnih implementacija čini sigurnost RSA algoritma i danas aktuelnom temom u kontekstu side-channel napada.

Pored demonstracije ranjivosti, rad evaluira i kontramjere: constant-time implementaciju koja potpuno eliminiše leak, RSA blinding čija ograničenja demonstriraju razliku između branch-zavisnih i operand-zavisnih timing leaka, te utjecaj kompajlerskih optimizacija na sigurnost.

1.1.1 Doprinosa rada

Konkretni doprinosi ovog rada su:

- Statistička kvantifikacija timing leaka na modernom procesoru (Intel i5-1335U, Raptor Lake), uključujući šest komplementarnih statističkih testova, bootstrap intervale povjerenja i analizu statističke snage (*power analysis*).
- Potpuna slijepa rekonstrukcija 64-bitnog privatnog RSA ključa isključivo iz timing mjerenja sa 100% tačnošću, pri čemu napadački kod nikada nije imao direktan pristup vrijednosti tajnog ključa.
- Empirijska evaluacija tri kontramjere: constant-time implementacije, RSA blindinga i utjecaja kompajlerskih optimizacija.
- Demonstracija Flush+Reload cache napada kao kvalitativno drugačijeg mehanizma detekcije s višestruko jačim signalom.

1.2 Struktura rada

Poglavlje 2 obrađuje teorijske osnove neophodne za razumijevanje eksperimenta: RSA kriptosistem, square-and-multiply algoritam, klasifikaciju timing napada u kontekstu relevantne literature, te statističke metode korištene u analizi (t-test, Mann-Whitney U, Cohen-ov d, bootstrap intervali povjerenja, power analiza).

Poglavlje 3 dokumentuje eksperimentalnu metodologiju: obje varijante modularne eksponencijacije (*naive* i *constant-time*), mehanizam preciznog mjerenja CPU ciklusa pomoću RDTSCP instrukcije, te dizajn eksperimenta zasnovan na uparenom pristupu (*paired approach*).

Poglavlje 4 predstavlja rezultate sedam provedenih eksperimenata: detekcija timing leaka, averaging napad, rekonstrukcija 64-bitnog ključa, sistematska multi-bit analiza, utjecaj kompajlerskih optimizacija, evaluacija RSA blindinga kao kontramjere, te Flush+Reload cache napad. Poglavlje se završava diskusijom o značaju rezultata.

Poglavlje 5 sumira ključna postignuća, formuliše preporuke za sigurniju implementaciju i navodi ograničenja istraživanja te moguće pravce daljeg rada.

U Prilogu A dokumentovana je konfiguracija eksperimentalnog okruženja i prikazani su ključni assembly isječci koji verificiraju ispravnost obje implementacije na razini mašinskog koda.

Poglavlje 2

Teorijske osnove

2.1 RSA kriptosistem

RSA kriptosistem, koji su 1978. godine predložili Rivest, Shamir i Adleman [1], je prvi javno objavljeni praktično primjenjeni sistem enkripcije s javnim ključem.¹ Temelji se na asimetričnom paru ključeva: javnom ključu koji se slobodno distribuira i privatnom ključu koji ostaje tajan.

2.1.1 Generisanje ključeva

Generisanje RSA ključeva odvija se u sljedećim koracima:

1. Odaberu se dva velika prosta broja p i q , pri čemu je $p \neq q$.
2. Računa se *modulus* $n = p \cdot q$.
3. Računa se Eulerova phi-funkcija (funkcija totijenta): $\varphi(n) = (p - 1)(q - 1)$.
4. Bira se javni eksponent e takav da je $1 < e < \varphi(n)$ i $\gcd(e, \varphi(n)) = 1$.
5. Privatni eksponent d je modularni inverz od e po modulu $\varphi(n)$:

$$d \cdot e \equiv 1 \pmod{\varphi(n)} \quad (2.1)$$

Javni ključ je par (e, n) , a privatni ključ je d (zajedno s n).

2.1.2 Šifriranje i dešifriranje

Za poruku M ($0 \leq M < n$), šifriranje se vrši eksponenciranjem s javnim ključem:

$$C = M^e \pmod{n} \quad (2.2)$$

Dešifriranje se vrši eksponenciranjem šifrirane poruke C s privatnim ključem:

$$M = C^d \pmod{n} \quad (2.3)$$

Ispravnost dešifriranja garantuje Eulerova teorema: $M^{e \cdot d} \equiv M^{k\varphi(n)+1} \equiv M \pmod{n}$ [1].

¹Diffie i Hellman su 1976. predložili teorijski okvir za razmjenu ključeva javnim kanalom, ali nisu specificirali konkretan sistem enkripcije. Clifford Cocks iz britanskog GCHQ-a razvio je ekvivalent RSA-a 1973, ali je to ostalo klasificirano do 1997. [6]

2.1.3 Sigurnost RSA-a

Sigurnost RSA-a se zasniva na pretpostavci da je faktORIZACIJA produkta dva velika prosta broja $n = p \cdot q$ praktično neizvodljiva. Poznati najbrži algoritam faktORIZACIJE za klasične računare (General Number Field Sieve) ima subekspnencijalnu složenost $O(\exp(c(\ln n)^{1/3}(\ln \ln n)^{2/3}))$ [7]. Za modulus od 2048 bita koji se danas preporučuje, ovo je računski nedostižno s poznatim klasičnim računarskim resursima.²

Međutim, sigurnost algoritma ne implicira automatski sigurnost implementacije, i upravo tu nastaju side-channel ranjivosti koje su tema ovog rada.

2.2 Modularna eksponencijacija

Glavna operacija u RSA kriptosistemu je *modularna eksponencijacija*: izračunavanje M^d mod n za veliko d . Direktno uzastopno množenje M sa samim sobom d puta je računski neprihvatljivo: za $d \approx 2^{2048}$, to bi zahtijevalo 2^{2048} množenja. Umjesto toga, koristi se algoritam kvadriraj-i-množi, koji svodi kompleksnost na $O(\log_2 d)$ množenja.

2.2.1 Kvadriraj-i-množi algoritam

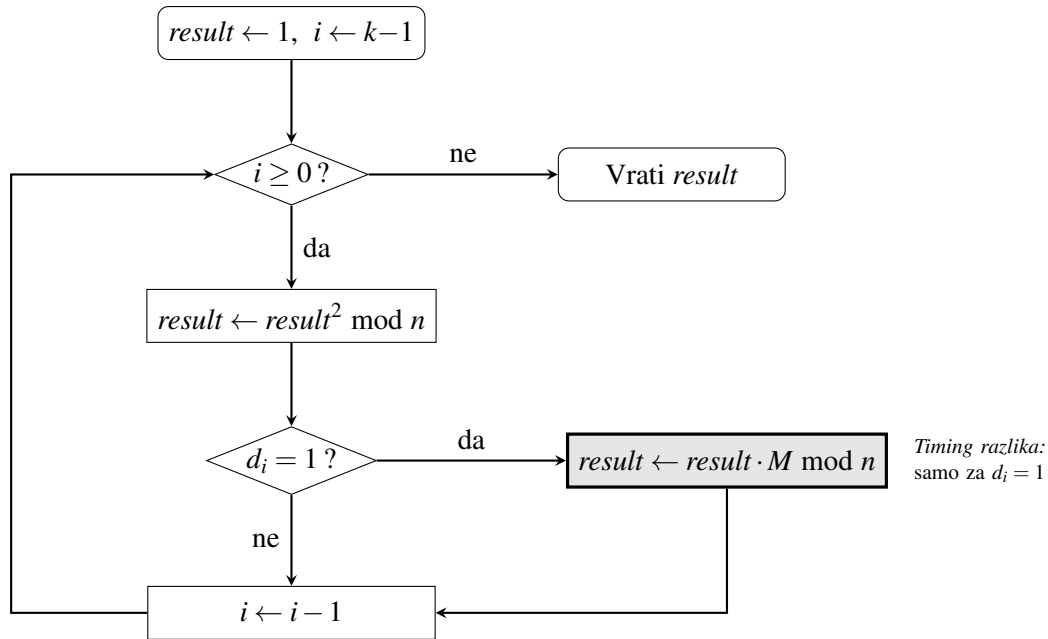
Neka su $d_{k-1}, d_{k-2}, \dots, d_1, d_0$ biti privatnog eksponenta d od MSB (najznačajnijeg) ka LSB (najmanje značajnom). Algoritam od lijeva do desno kvadriraj-i-množi [10] je sljedeći:

Program 2.1: Kvadriraj-i-množi algoritam (pseudokod)

```
result = 1
za i od k-1 do 0:
    result = result * result mod n
    ako je d[i] == 1:
        result = result * M mod n
vrati result
```

Algoritam radi k kvadriranja i tačno w množenja, gdje je w Hammingova težina eksponenta d (broj jediničnih bita). Za nasumičan 2048-bitni eksponent (koji je preporučen za korištenje u produkcijskim sistemima), očekivana vrijednost w je 1024, pa algoritam tipično izvodi oko 3072 modularne multiplikacije. Dijagram toka algoritma prikazan je na slici 2.1.

²Shorov kvantni algoritam može faktORIZOVATI RSA modulus u polinomijalnom vremenu [8], što je pokrenulo razvoj post-kvantnih kriptografskih standarda. NIST je 2024. finalizirao prve post-kvantne standarde (FIPS 203, 204, 205) [9]. Analiza u ovom radu ograničena je na klasično okruženje.



Slika 2.1: Dijagram toka kvadriraj-i-množi algoritma. .

Primjer

Za $d = 11$ (binarno: 1011) i $M = 5$, $n = 17$, tok algoritma je:

Bit $d[i]$	Kvadriranje	Množenje	Međurezultat
1	$1^2 = 1$	$1 \cdot 5 = 5$	5
0	$5^2 = 25 \equiv 8$	/	8
1	$8^2 = 64 \equiv 13$	$13 \cdot 5 = 65 \equiv 14$	14
1	$14^2 = 196 \equiv 9$	$9 \cdot 5 = 45 \equiv 11$	11

Rezultat: $5^{11} \bmod 17 = 11$, što se može verifikovati računski.

2.2.2 Vremenska ranjivost algoritma

Ključno zapažanje, ilustrirano dijagramom toka na slici 2.1, je da algoritam iz isječka koda 2.1 **ne izvodi isti broj operacija za sve vrijednosti bita eksponenta**:

- Za $d[i] = 0$: izvodi se **samo kvadriranje** (jedna modularna multiplikacija)
- Za $d[i] = 1$: izvodi se **kvadriranje i množenje** (dvije modularne multiplikacije)

Svaka dodatna modularna multiplikacija dodaje mjerljiv prosječan trošak u CPU ciklusima. Razlika u trajanju po bitu jednaka je prosječnom trajanju jedne `mulmod` operacije, i to je izvor timing side-channel ranjivosti koju analiziramo u ovom radu.

2.3 Side-channel napadi

2.3.1 Definicija i klasifikacija

Side-channel napad iskorištava informacije koje "cure" iz fizičke realizacije kriptografskog sistema, a ne iz matematičke slabosti algoritma [11]. Za razliku od kriptanalize koja napada matematičku strukturu, side-channel napadi mjere fizički mjerljive veličine koje povezuju sa tajnim podacima. Glavne kategorije side-channel napada su:

Timing napadi. Mjeri se trajanje kriptografskih operacija. Signal može biti apsolutno trajanje, relativna razlika između operacija ili statistička distribucija trajanja [2].

Analiza potrošnje energije (*power analysis*). Analizira se potrošnja uređaja na kojem se izvršava kriptografski sistem. Simple power analysis (SPA) vizualno analizira pojedinačne tragove strujne potrošnje, a differential power analysis (DPA) koristi statističku analizu većeg broja tragova s različitim ulaznim podacima [12].

Elektromagnetski napadi. Mjeri se elektromagnetna emisija uređaja tokom kriptografske operacije i izvodi se analiza signala. Primjenjivi su i bez fizičkog kontakta s uređajem [13].

Cache napadi. Iskorištavaju zajednički Last Level Cache (LLC) na višezvezganim procesorima, gdje napadač može mjeriti vrijeme pristupa cache memoriji nakon njenog brisanja [14]. Detaljno su opisani u sekciji 2.3.4.

Akustični napadi. Vršiti se analiza zvukova koje emituju komponente računara tokom rada [15].

2.3.2 Kocher-ov timing napad (1996)

Paul Kocher je 1996. godine demonstrirao da naivne implementacije Diffie-Hellman, RSA, DSS i sličnih protokola otkrivaju informaciju o privatnom ključu kroz trajanje operacije [2].

Osnovna ideja napada je sljedeća: napadač bira skup mjerenih poruka i za svaku bilježi trajanje kriptografske operacije. Mjerenja se grupišu prema hipotezi o vrijednosti pojedinog bita ključa. Ako je hipoteza ispravna, grupe mjerenja imat će statistički različite distribucije, jer je tajni bit konzistentno utjecao na trajanje. Ako je hipoteza pogrešna, distribucije će biti identične (šum). Napad zahtijeva da napadač može:

1. Slati proizvoljne poruke na kriptografsku funkciju, i
2. Mjeriti trajanje operacije s dovoljnom preciznošću.

U originalnom radu, Kocher je ciljao smart kartice koje nisu imale nikakvu zaštitu od timing napada. Pokazao je da je moguće rekonstruisati privatni ključ uz nekoliko stotina do nekoliko hiljada mjerenja.

2.3.3 Brumley-Boneh mrežni napad (2003)

Brumley i Boneh su 2003. godine proširili Kocher-ov napad na mrežni scenarij [4]. Napadali su stvarni OpenSSL server na lokalnoj mreži, uzimajući u obzir mrežnu latenciju kao dodatni izvor šuma. Korištenjem tehnika kao što su:

- Slanje velikog broja zahtjeva za svaki bit,
- Pažljiv odabir poruka koji pojačava timing razliku, i
- Statistička analiza razlika u latenciji

uspjeli su rekonstruisati faktor RSA privatnog ključa. Ovaj napad je imao direktan praktičan utjecaj: OpenSSL je u verziji 0.9.7 implementirao RSA blinding kao kontramjeru.

2.3.4 Cache timing side-channel napadi

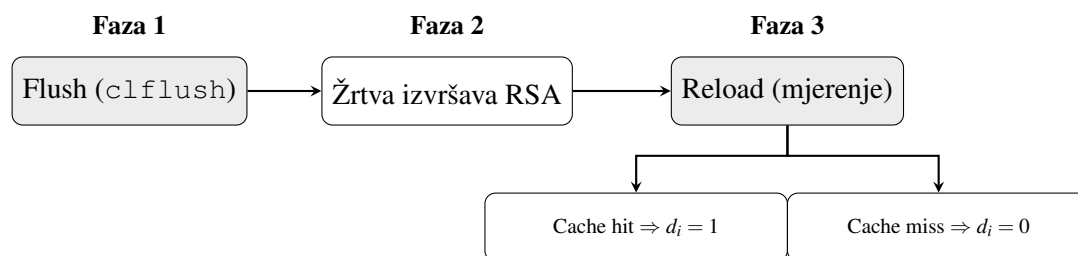
Za razliku od timing napada koji mjere ukupno trajanje operacije, cache timing side-channel napadi prate fizički pristup specifičnim memorijskim adresama kroz zajednički CPU cache. Na modernim višezvezganim procesorima sva jezgra dijele LLC (Last Level Cache, tipično L3) i ovo dijeljenje, kombinovano s činjenicom da procesi koji koriste istu dijeljenu biblioteku (`.so` fajl) dijele i fizičke memorijske stranice te biblioteke, stvara tajni kanal između napadača i žrtve.

Flush+Reload

Flush+Reload napad [14] odvija se u tri koraka koji se ponavljaju:

1. **Flush:** Napadač uklanja specifičnu cache liniju koristeći `clflush` instrukciju. Ova instrukcija je dostupna korisniku bez privilegija i garantovano uklanja adresu iz svih nivoa cache-a.
2. **Čekanje:** Napadač čeka dovoljno dugo da žrtva izvrši ciljanu operaciju.
3. **Reload:** Napadač mjeri kašnjenje ponovnog učitavanja (reload) te cache linije. Kratka latencija ($\approx$ L1/L2/LLC latencija, npr. 100 do 400 ciklusa) znači da je žrtva pristupila adresi, odnosno *cache hit*, a duga latencija ($\approx$ DRAM latencija, npr. 500 do 1000 ciklusa) znači da žrtva nije pristupila adresi, odnosno *cache miss*.

Slika 2.2 ilustrira ove tri faze i dva moguća ishoda.



Slika 2.2: Faze Flush+Reload napada

Ključni preduvjet je **dijeljena memorija**: i napadač i žrtva moraju imati mapiranu istu fizičku stranicu. Ovo je automatski zadovoljeno za dijeljene biblioteke (npr. `libc.so`) gdje operativni sistem mapira jednu kopiju koda u fizičku memoriju, a oba procesa vide iste fizičke cache linije. U kontekstu RSA napada, napadač cilja funkciju uslovnog množenja koja se poziva *samo* kada je bit eksponenta jednak 1. Kada žrtva izvrši dekriptovanje pri čemu je ciljani bit eksponenta jednak 1, CPU učitava cache liniju te funkcije i napadač detektuje cache hit. Kada je bit jednak 0, funkcija se ne poziva i napadač detektuje cache miss.

Prime+Probe

Prime+Probe napad ne zahtijeva dijeljenu memoriju između napadača i žrtve, što ga čini primjenjivim čak i između virtualnih mašina. Napadač *puni* (Prime) skup cache linija (cache set) vlastitim podacima, čeka da žrtva izvrši operaciju, a potom mjeri kašnjenje ponovnog pristupa (Probe) tim linijama. Ako je žrtva pristupila adresama koje mapiraju u iste cache linije, napadačeve linije su istisnute, što rezultira sporijim pristupom. Ovaj napad je složeniji za implementaciju i daje slabiji signal od Flush+Reload, ali funkcioniše i u scenarijima bez dijeljene memorije.

2.3.5 Kontramjera: Constant-time implementacija

Constant-time implementacija je tehnika koja garantuje da trajanje kriptografske operacije ne zavisi od tajnih podataka. Princip je eliminacija uslovnog grananja (*branchless programming*, programiranje bez grananja): umjesto `if-else` konstrukcije koja izvršava različit broj operacija za različite vrijednosti bita, uvijek se izvršavaju obje operacije (i kvadriranje i množenje), a rezultat se bira pomoću *conditional move* (`cmov`) instrukcije ili aritmetičkog maskiranja, bez ikakvog skoka u kontrolnom toku. Na taj način, CPU izvršava identičan broj operacija i pristupa istim memorijskim adresama bez obzira na vrijednost bita, čime se eliminira i timing i cache side-channel. Konkretna implementacija opisana je u sekciji 3.4.3.

2.3.6 Kontramjera: RSA blinding

RSA blinding je kontramjera uvedena u OpenSSL nakon Brumley-Boneh napada [4]. Zove se blinding ili zasljepljivanje jer "krije" podatke koji se obrađuju od vanjskih posmatrača. Princip je nasumičan odabir ulaza u eksponencijaciji:

1. Odabere se nasumičan r takav da $\gcd(r, n) = 1$.
2. Izračuna se blinded ulaz: $M' = M \cdot r^e \bmod n$.
3. Eksponencijacija: $S' = (M')^d \bmod n$.
4. Unblinding: $S = S' \cdot r^{-1} \bmod n = M^d \bmod n$.

Budući da napadač ne poznaje r , ne može povezati svoje poznate ulaze s internim međuvrijednostima. U realnom RSA s višepreciznom aritmetikom, ovo maskira operand-zavisne vremenske varijacije. Međutim, blinding ne mijenja eksponent d i kontrolni tok ostaje identičan, što ima implikacije za branch-zavisna vremenska curenja analizirana u poglavlju 4.

2.3.7 Model u ovom radu

U ovom radu analiziran je lokalni timing napad s precizno definisanim modelom:

Napadač. Lokalni korisnik na istoj mašini, bez privilegija superkorisnika.

Mogućnosti napada. Pokretanje vlastitog koda i mjerenje trajanja kriptografskih operacija u CPU ciklusima. U ovom radu koriste se dva različita modela napadača, ovisno o eksperimentu:

- **Eksperimenti 1, 2, 4, 5 i 6 (karakterizacija signala):** Napadač može pozvati modularnu ekspanencijaciju s proizvoljno odabranim eksponentima, uključujući parove koji se razlikuju samo u jednom bitu. Tajni ključ S nije relevantan za ove eksperimente jer se napad bavi mjerenjem timing razlike između eksponenata koje napadač sam konstruiše.
- **Eksperiment 3 (slijepa rekonstrukcija):** Napadač ne poznaje vrijednost tajnog ključa S i nikada je ne vidi. Jedino što mu je dostupno jesu izmjerena trajanja: koliko dugo traje ekspanencijacija s tajnim ključem za odabranu bazu M , te koliko bi trajala da je samo i -ti bit tog ključa bio obrnut i tu izmjenu sistem primjenjuje interno, na skrivenom S . Iz razlike tih trajanja napadač zaključuje koja je bila izvorna vrijednost bita i tako rekonstruiše ključ bit po bit [2].

Ograničenja napada. Nema pristupa memoriji procesa žrtve; nema mrežnog pristupa; nema fizičkog pristupa hardveru.

Tajna. Privatni eksponent d (64-bitni u ovom eksperimentu).

Javno poznato. Baza M , modulus n , broj bita eksponenta.

Model prijetnji za Eksperiment 3 odgovara Kocher-ovom originalnom scenariju [2]: napadač ne zna S i rekonstruiše ga isključivo iz izmjerenih trajanja. Ovaj eksperiment demonstrira mehanizam curenja informacija pod idealiziranim lokalnim uslovima. Dok u realnom scenariju napadač bi imao ograničeniji pristup, npr. samo mrežni timing kao kod Brumley i Boneh [4], što bi zahtijevalo više mjerenja, ali bi temeljni mehanizam ostao isti.

Ovakav pristup je praktično primjenjiv svuda gdje postoji API za dekriptovanje: na smart karticama i HSM (*Hardware Security Module*) uređajima, u dijeljenim kriptografskim servisima, te u TLS 1.2 i starijim handshake protokolima s RSA ključnom razmjenom. U svim tim slučajevima napadač može slati pažljivo odabrane ulaze i mjeriti koliko traje svaki odgovor, bez ikakvog direktnog uvida u privatni ključ.

2.4 Statističke metode u analizi side-channel napada

2.4.1 Problem signala i šuma

Trajanje kriptografskih operacija na modernom CPU-u nije konstantno. Brojni faktori doprinose varijabilnosti:

- **Promašaji cache-a:** Pristup memoriji koja nije u L1/L2/L3 cache-u dodaje stotine ciklusa latencije.
- **Predikcija grananja:** Procesor predviđa smjer skokova; pogrešna predikcija dodaje kaznu od 10 do 20 ciklusa.
- **OS preempcija:** Operativni sistem može prekinuti izvršavanje procesa u bilo kom trenutku (context switch).
- **TLB i memorijske barijere:** Invalidacija prijevodnog međuspremnik (TLB) i memorijske barijere dodaju varijabilno kašnjenje.

- **Toplotno ograničavanje:** Pregrijani procesor smanjuje frekvenciju, povećavajući trajanje.

Ukupni šum σ u distribuciji trajanja obično je daleko veći od signala $\mu_1 - \mu_0$ koji odgovara jednoj mulmod operaciji. Omjer signal-šum (SNR) po jednom mjerenju definisan je kao:

$$\text{SNR} = \frac{|\mu_1 - \mu_0|}{\sigma_{\text{pool}}} \quad (2.4)$$

gdje su μ_1 i μ_0 srednje vrijednosti trajanja za bit = 1 i bit = 0, a σ_{pool} je ujedinjena standardna devijacija. SNR je ključna metrika jer određuje **izvodljivost napada**: ako je SNR visok, jedno mjerenje je dovoljno za detekciju bita; ako je nizak, napadač mora prikupiti mnogo mjerenja i usrednjiti ih. Ova veličina je po formi identična apsolutnoj vrijednosti Cohen-ovog d (jednačina 2.8): $\text{SNR} = |d|$. Obje metrike se koriste jer d ima standardiziranu interpretaciju veličine efekta (mali/srednji/veliki), dok SNR direktno određuje minimalan broj mjerenja $K_{\min}$ potrebnih za uspješan napad. U ovom eksperimentu $\text{SNR} = |d| \approx 0,134$, što znači da je signal oko 7,5 puta manji od šuma i da je pojedinačno mjerenje nepouzđano.

2.4.2 Usrednjavanje i centralna granična teorema

Temeljni princip koji omogućava timing napad uprkos niskom SNR-u po jednom mjerenju je *usrednjavanje* (averaging). Centralna granična teorema garantuje da srednja vrijednost K nezavisnih mjerenja konvergira ka pravoj sredini bez obzira na oblik distribucije, što je od ključne važnosti jer distribucije CPU trajanja nisu normalne. Ako napadač prikupi K nezavisnih mjerenja za isti bit i usrednji ih, standardna devijacija usrednjenog procjenitelja opada prema:

$$\sigma_K = \frac{\sigma}{\sqrt{K}} \quad (2.5)$$

Signal ostaje konstantan ($\mu_1 - \mu_0$), a šum opada kao $1/\sqrt{K}$. Minimalan broj mjerenja $K_{\min}$ potrebnih da $\text{SNR}_K > r$ (tj. da signal bude r puta veći od šuma) je:

$$K_{\min} = \left(\frac{r}{\text{SNR}} \right)^2 \quad (2.6)$$

Za 95% tačnost klasifikatora (*threshold*) na normalnim distribucijama, potrebno je $r \approx 2$, što za $\text{SNR} = 0,134$ daje $K_{\min} = (2/0,134)^2 \approx 223$.

2.4.3 Welch-ov t-test

U našem eksperimentu, Welch-ov t-test [16] služi za testiranje nulte hipoteze $H_0 : \mu_0 = \mu_1$, odnosno da ne postoji razlika u trajanju operacije između slučajeva bit=0 i bit=1. Odbacivanje H_0 potvrđuje da timing razlika postoji i da je statistički značajna, a ne rezultat slučajnog šuma. Također, Welch-ova varijanta ne pretpostavlja jednakost varijansi grupa. Ovo je bitno u kontekstu vremenskih mjerenja jer operacije za bit=0 (samo kvadriranje) i bit=1 (kvadriranje i množenje) mogu imati različite obrasce pristupa memoriji i cache-u, što dovodi do različitih varijansi. Statistika testa je:

$$t = \frac{\bar{X}_0 - \bar{X}_1}{\sqrt{s_0^2/n_0 + s_1^2/n_1}} \quad (2.7)$$

gdje su $\bar{X}_i$, s_i^2 i n_i redom srednja vrijednost, varijansa i broj uzoraka za grupu i .

2.4.4 Mann-Whitney U test

Mann-Whitney U test [17] koristi se u ovom radu kao dopunska provjera rezultata Welch-ovog t-testa. Dok t-test pretpostavlja da su podaci normalno distribuirani, Mann-Whitney U test ne zahtijeva nikakvu pretpostavku o obliku distribucije. Ovo je posebno važno za distribucije trajanja CPU operacija, koje tipično imaju dugačak desni rep uzrokovan povremenim cache promašajima i OS preempcijama, te stoga nisu Gaussovske.

Test se zasniva na rangiranju svih uzoraka iz obje grupe zajedno i poređenju pozicija rangova između grupa. Ako mjerenja za bit=1 konzistentno zauzimaju više rangove (duže traju), test će to detektovati. Korištenjem i parametarskog (Welch) i neparametarskog (Mann-Whitney) testa, dobijamo potvrdu da uočena razlika nije posljedica distribucijskih pretpostavki.

2.4.5 Cohen-ova mjera veličine efekta

Statistička značajnost (p-vrijednost) sama po sebi nije dovoljna za procjenu izvodljivosti napada. Uz dovoljno veliki uzorak, bilo koja razlika, pa i trivijalno mala, postaje statistički značajna. Cohen-ov d [18] rješava ovaj problem mjereći **praktičnu veličinu** razlike između dvije grupe, nezavisno od broja uzoraka:

$$d = \frac{\mu_1 - \mu_0}{\sigma_{\text{pool}}} \quad (2.8)$$

gdje je $\sigma_{\text{pool}} = \sqrt{((n_0 - 1)s_0^2 + (n_1 - 1)s_1^2)/(n_0 + n_1 - 2)}$ ujedinjena standardna devijacija. Standardna interpretacija [18]: $|d| < 0,2$ mali efekat; $0,2 \leq |d| < 0,5$ srednji efekat; $0,5 \leq |d| < 0,8$ srednje-veliki efekat; $|d| \geq 0,8$ veliki efekat.

U kontekstu timing napada, Cohen-ov d kvantificira koliko su distribucije trajanja za bit=0 i bit=1 razdvojene. Mali d (kao naš $\approx 0,134$) znači da se distribucije jako preklapaju i da napadač ne može pouzdano klasificirati jedan uzorak. Međutim, mali d ne znači da napad ne funkcioniše: uz usrednjavanje K mjerenja, efektivni d raste kao $d\sqrt{K}$, pa i mali efekt postaje detektabilan s dovoljno mjerenja. Cohen-ov d direktno određuje i minimalnu veličinu uzorka za napad (jednačina 2.6).

2.4.6 Upareni t-test

U ovom eksperimentu, mjerenja su prirodno sparena: za istu baznu vrijednost mjere se trajanja s bit=0 i bit=1 pod istim eksperimentalnim uslovima (stanje cache-a, frekvencija procesora, pozadinski procesi). Upareni t-test (paired t-test) iskorištava ovu strukturu podataka. Umjesto da poredi dvije nezavisne grupe, test analizira razlike unutar svakog para, čime se eliminira varijansa koja potiče od razlika između mjerenja (npr. različito opterećenje sistema u različitim trenucima). Statistika testa je:

$$t = \frac{\bar{D}}{s_D/\sqrt{n}} \quad (2.9)$$

gdje je $\bar{D}$ srednja vrijednost parnih razlika $D_i = T_{i,1} - T_{i,0}$, s_D standardna devijacija razlika, a n broj parova. Ključna prednost je što je s_D tipično znatno manji od σ_{pool} jer se faktori šuma koji su zajednički za oba mjerenja u paru međusobno poništavaju, čime se efektivno povećava statistička snaga testa.

2.4.7 Bootstrap interval povjerenja

Parametarski intervali povjerenja pretpostavljaju normalnost distribucije, što za iskošene distribucije CPU trajanja nije opravdano. Bootstrap metoda [19] procjenjuje interval povjerenja bez ikakvih distribucijskih pretpostavki. Postupak: iz originalnog uzorka uzima se B nasumičnih uzoraka s ponavljanjem (resampling), za svaki se računa statistika interesa (u našem slučaju razlika sredina), a granice intervala su odgovarajući procenti empiričke distribucije.

U kontekstu ovog rada, bootstrap interval povjerenja za $\Delta = \mu_1 - \mu_0$ daje raspon prihvatljivih vrijednosti za pravu razliku u trajanju. Ako 95% interval ne sadrži nulu, možemo s visokom pouzdanošću tvrditi da razlika postoji. Koristi se $B = 10000$ iteracija.

2.4.8 Power analiza

Power analiza je ključan korak u planiranju eksperimenta jer određuje **minimalan broj uzoraka** $N_{\min}$ potreban za pouzdanu detekciju efekta dane veličine, prije nego što se eksperiment uopće provede. Za timing napad, ovo direktno odgovara na pitanje: koliko mjerenja napadač mora prikupiti da bi napad bio izvodljiv? Za dvostrani t-test [18]:

$$N_{\min} = \left(\frac{z_{1-\alpha/2} + z_{1-\beta}}{d} \right)^2 \quad (2.10)$$

gdje je d Cohen-ov d , z kvantili standardne normalne distribucije, α vjerovatnoća lažnog pozitiva, a $(1 - \beta)$ željena statistička snaga (vjerovatnoća detekcije stvarnog efekta). Za $\alpha = 0,05$, snaga = 0,80 i $d = 0,134$: $N_{\min} \approx 438$ po grupi. Ovim se zatvara krug između veličine efekta i praktičnosti napada: mali efekat zahtijeva više mjerenja, što znači duži napad.

2.5 Hardverske karakteristike relevantne za eksperiment

2.5.1 Time Stamp Counter (TSC)

Za precizno mjerenje trajanja operacija koristi se TSC (Time Stamp Counter), 64-bitni registar koji se čita instrukcijama RDTSC i RDTSCP. Na procesoru Intel i5-1335U (Raptor Lake), TSC ima garancije `constant_tsc` (raste po konstantnoj stopi neovisno o frekvenciji procesora) i `nonstop_tsc` (ne pauzira u stanjima štednje energije (*power-saving*)) [20], čime se osigurava pouzdana baza za mjerenje.

Na *out-of-order* procesorima, instrukcije mogu biti izvršavane izvan programskog redoslijeda. Intel preporučuje [21] serijalizaciju mjernog prozora: `CPUID` → `RDTSC` na početku, `RDTSCP` → `CPUID` na kraju. `CPUID` djeluje kao barijera koja garantuje povlačenje (*retirement*) svih prethodnih instrukcija, čime se osigurava da mjerni prozor obuhvata tačno ciljanu operaciju.

2.5.2 Izvori šuma u mjerenjima

Izvori šuma u timing mjerenjima (opisani u sekciji 2.4) zahtijevaju primjenu mjera za kontrolu eksperimentalnih uslova: fiksiranu frekvenciju procesora, CPU afinitet i zagrijavanje cache-a. Detalji ovih mjera opisani su u poglavlju 3.

Sa ovim teorijskim i statističkim temeljima uspostavljenim, naredno poglavlje opisuje eksperimentalni dizajn i implementacijske odluke koje prevode ove principe u mjerljive i ponovljive rezultate.

Poglavlje 3

Implementacija i eksperimentalni dizajn

3.1 Formalni eksperimentalni okvir

3.1.1 Hipoteze

Eksperiment testira sljedeće statističke hipoteze na razini značajnosti $\alpha = 0,05$:

H_0 (**nulta hipoteza**): $\mathbb{E}[T \mid d[i] = 0] = \mathbb{E}[T \mid d[i] = 1]$

Trajanje modularne eksponencijacije ne zavisi od vrijednosti bita eksponenta.

H_1 (**alternativna hipoteza**): $\mathbb{E}[T \mid d[i] = 0] \neq \mathbb{E}[T \mid d[i] = 1]$

Postoji statistički značajna razlika u trajanju između slučajeva bit=0 i bit=1.

Odbacivanje H_0 u korist H_1 pruža statistički dokaz o curenju informacija.

3.1.2 Eksperimentalne varijable

Nezavisna varijabla: Vrijednost ciljanog bita privatnog eksponenta ($d[i] \in \{0, 1\}$).

Zavisna varijabla: Trajanje modularne eksponencijacije u CPU ciklusima (T).

Kontrolisane varijable: Baza (M), modulus (n), Hammingova težina eksponenta izvan pozicije i , CPU frekvencija, te CPU afinitet.

3.2 Testna platforma i priprema okruženja

3.2.1 Specifikacije testne platforme

Eksperiment je proveden na sljedećoj platformi:

Tabela 3.1: Specifikacije testne platforme

Komponenta	Specifikacija
Računar	Lenovo ThinkPad T14 Gen 4
Procesor	Intel Core i5-1335U (Raptor Lake, 13. gen.) [22]
Arhitektura	Hibridna: 2 P-core + 4 E-core (10 fizičkih jezgri i 12 niti)
Cache	L1/L2 privatni po jezgrama, L3: 12 MB Intel Smart Cache
RAM	16 GB LPDDR5
Operativni sistem	Ubuntu Linux 24.04 LTS
Kernel	6.17.0-19-generic (x86_64)
Kompajler	GCC 13.3

3.2.2 Kontrola eksperimentalnih uslova

Za standardizaciju mjerenja primijenjene su sljedeće mjere izolacije šuma koje se koriste u literaturi timing side-channel napada [2, 4]:

- **Fiksna frekvencija procesora:** Isključen Turbo Boost i postavljen performance governor (koji zamjenjuje podrazumijevani powersave governor s `intel_pstate` upravljačem) kako bi TSC ciklusi odgovarali konstantnoj količini procesorskog rada.
- **CPU afinitet:** Proces fiksiran na P-core 0 (`sched_setaffinity`) čime se eliminira varijansa uzrokovana migracijom između jezgri s različitim latencijama.
- **Zagrijavanje:** Prvih 10 000 iteracija se odbacuje radi stabilizacije cache memorije, branch prediktora i termičkog stanja procesora.

Izmjerena TSC frekvencija iznosi $\approx 2,101$ GHz, što odgovara nominalnoj frekvenciji i potvrđuje invarijantnost TSC-a. Detalji konfiguracije (komande, parametri) dati su u Prilogu A.

3.3 Implementacija mjerenja vremena

3.3.1 RDTSC s CPUID serijalizacijom

Za precizno mjerenje CPU ciklusa implementirano je mjerenje preporučeno od strane Intela [21]:

Program 3.1: Serijalizovano čitanje TSC – početak mjerenja

```
static inline uint64_t tsc_start(void) {
    uint32_t lo, hi;
    __asm__ __volatile__ (
        "CPUID\n\t"
        "RDTSC\n\t" /* Citaj TSC u EDX:EAX */
        "mov_%%edx,_%0\n\t"
        "mov_%%eax,_%1\n\t"
        : "=r"(hi), "=r"(lo)
        :: "%rax", "%rbx", "%rcx", "%rdx"
    );
}
```

```
    return ((uint64_t)hi << 32) | (uint64_t)lo;
}
```

Program 3.2: Serijalizovano čitanje TSC – kraj mjerenja

```
static inline uint64_t tsc_end(void) {
    uint32_t lo, hi;
    __asm__ __volatile__ (
        "RDTSCP\n\t" /* Citaj TSC */
        "mov_%%edx,_%0\n\t"
        "mov_%%eax,_%1\n\t"
        "CPUID\n\t" /* Sprijeci premještanje kasnijeg koda */
        : "=r"(hi), "=r"(lo)
        :: "%rax", "%rbx", "%rcx", "%rdx"
    );
    return ((uint64_t)hi << 32) | (uint64_t)lo;
}
```

Instrukcija CPUID djeluje kao barijera kojom se garantuje potpuno povlačenje (*retirement*) svih prethodnih instrukcija iz *out-of-order* izvršnog stroja [21]. RDTSCP analogno osigurava da sve mjerene instrukcije dostignu fazu povlačenja prije nego što se vrijednost TSC-a uzorkuje. CPUID koji slijedi iza RDTSCP sprječava spekulativno izvlačenje narednih instrukcija u mjerni prozor.

3.3.2 Ograničenja kompajlera

Da bi se spriječilo da kompajler premjesti instrukcije unutar prozora u kojem mjerimo vrijeme ili ukloni računanje, koriste se sljedeće direktive:

Program 3.3: Barijere kompajlera za zaštitu mjernog prozora

```
/* Sprijeci premještanje instrukcija od strane kompajlera */
#define COMPILER_BARRIER()
    __asm__ __volatile__ (" : : : \"memory\" )

/* Sprijeci eliminaciju rezultata */
#define DO_NOT_OPTIMIZE(x) \
    __asm__ __volatile__ (" : : \"r,m\"(x) : \"memory\" )
```

3.4 Implementacija RSA modularne eksponencijacije

3.4.1 Modularna multiplikacija

Za izračunavanje $a \cdot b \bmod m$ gdje su $a, b < 2^{61}$ (modulus je $2^{61} - 1$), produkt $a \cdot b$ može biti do 2^{122} , što se ne može pohraniti u 64-bitni tip. Zbog toga se koristi GCC ekstenzija `__uint128_t`:

Program 3.4: Modularna multiplikacija bez prekoračenja (*overflow*)

```
static inline uint64_t mulmod(uint64_t a, uint64_t b, uint64_t
m) {
    return (uint64_t) (
        ((__uint128_t)a * (__uint128_t)b) % (__uint128_t)m
    );
}
```

GCC prevodi ovo u jednu MULQ instrukciju (64-bitno množenje koje daje 128-bitni rezultat u RDX:RAX) [20], a zatim DIVQ za modulo.

3.4.2 Ranjiva implementacija

Ranjiva implementacija koristi eksplicitno grananje po bitu eksponenta:

Program 3.5: Ranjiva square-and-multiply implementacija

```
__attribute__((noinline))
__attribute__((optimize("O0")))
uint64_t mod_exp_vulnerable(
    uint64_t base, uint64_t exp,
    uint64_t mod, int num_bits) {
    uint64_t result = 1ULL;
    base = base % mod;
    for (int i = num_bits - 1; i >= 0; i--) {
        result = mulmod(result, result, mod);
        if ((exp >> i) & 1) {
            result = mulmod(result, base, mod);
        }
    }
    return result;
}
```

Atribut `optimize("O0")` korišten je kao mjera opreza da se osigura izvršavanje grananja. Eksperiment 5 (sekcija 4.5) naknadno potvrđuje da GCC ne generiše CMOV ni za jedan nivo optimizacije za ovaj specifičan kod, pa je atribut u ovom slučaju redundantan, ali je zadržan kao dobra praksa jer ponašanje kompajlera nije garantovano između verzija i platformi. *Sigurnost implementacije ne smije zavisiti od ponašanja kompajlera.*

3.4.3 Constant-time implementacija

Constant-time implementacija eliminiše grananje kroz bitovnu aritmetiku:

Program 3.6: Constant-time selekcija bez grananja

```
static inline uint64_t ct_select(
    uint64_t cond, uint64_t a, uint64_t b) {
    uint64_t mask = -(uint64_t)(cond & 1);
    return (a & mask) | (b & ~mask);
}
```

Program 3.7: Constant-time square-and-multiply

```
uint64_t mod_exp_constant_time(  
    uint64_t base, uint64_t exp,  
    uint64_t mod, int num_bits) {  
    uint64_t result = 1ULL;  
    base = base % mod;  
    for (int i = num_bits - 1; i >= 0; i--) {  
        uint64_t bit = (exp >> i) & 1;  
        result = mulmod(result, result, mod);  
        uint64_t mul = mulmod(result, base, mod);  
        result = ct_select(bit, mul, result);  
    }  
    return result;  
}
```

Ključna razlika: množenje `mulmod(result, base, mod)` *uvijek se izvršava*, bez obzira na vrijednost bita; `ct_select()` zatim bira koji rezultat koristiti, ali budući da ne sadrži ni jedan skok, CPU ne može razlikovati slučaj `bit=0` od slučaja `bit=1` po trajanju.

Ispravnost implementacije verifikovana je pregledom generisanog asemblerskog koda koristeći `objdump -d` (vidjeti Prilog A.7). Pregled asemblerskog koda `ct_select` funkcije potvrđuje da GCC generiše isključivo bitovne operacije bez skokova: `and`, `neg`, `not`, `or` instrukcije i nema nijedne uslovne instrukcije skoka (`je/jne`). Petlja u `mod_exp_constant_time` sadrži samo jedan skok koji kontrolira broj iteracija ($i \geq 0$), a ne vrijednost tajnog bita. Eksperiment 1 potvrđuje ovu analizu: razlika sredina od +0,14 ciklusa i $p = 0,84$ konzistentni su s hipotezom o odsustvu vremenskog curenja.

3.5 Dizajn eksperimenta prikupljanja podataka

3.5.1 Dizajn uparenih mjerenja

Ključna metodološka odluka u dizajnu eksperimenta je korištenje uparenog dizajna (*paired design*). Umjesto da se nasumično biraju eksponenti i klasifikuju po vrijednosti ciljanog bita, za svaku iteraciju generišu se *parovi* eksponenata:

1. Generiše se nasumičan 64-bitni bazni eksponent `base_exp`.
2. $\text{exp}_0 = \text{base_exp}$ s bitom i postavljenim na 0.
3. $\text{exp}_1 = \text{base_exp}$ s bitom i postavljenim na 1.

Budući da su exp_0 i exp_1 identični na svim bitovima osim na ciljanom bitu i , jedina razlika u trajanju *može biti samo* zbog bita i . Ovaj dizajn eliminiše faktor koji bi nastao da grupe imaju različite Hammingove težine (tj. različit ukupan broj jedinica), jer bi tada ukupno trajanje eksponencijacije variralo iz razloga koji nemaju veze s ciljanim bitom.

3.5.2 Nasumičan redoslijed

Redoslijed mjerenja exp_0 i exp_1 unutar svakog para se bira nasumično (50% vjerovatnoća zamjene) kako bi eliminisao pristranost nastalu od:

- Cache efekta (prvi poziv uvijek sporiji),
- Predviđanja grananja (CPU može biti brži za već viđene odluke).

3.5.3 Filtriranje rubnih vrijednosti

Mjerenja koja padaju izvan definisanog opsega se odbacuju:

- **Donja granica** (`MIN_CYCLES = 500`): Mjerenja kraća od 500 ciklusa fizički su nemoguća za 64-bitnu `mod_exp` operaciju i nagovještavaju grešku u mjerenju.
- **Gornja granica** (`OUTLIER_THRESHOLD = 50000`): Mjerenja duža od 50 hiljada ciklusa ukazuju na prekid konteksta (*context switch*) tokom mjerenja.

3.5.4 Parametri eksperimenta

Tabela 3.2: Parametri eksperimenta

Parametar	Vrijednost
Modulus n	$2^{61} - 1 = 2\,305\,843\,009\,213\,693\,951$ (Mersenne prost)
Baza M	1 234 567 890 123 456 789
Broj bita eksponenta	64
Ciljani bit u Eksperimentu 1	16
Broj parova mjerenja	500 000 po implementaciji
Iteracije zagrijavanja	10 000
PRNG seed	0xABCDEF1234567890

Mersenne prost broj $2^{61} - 1$ odabran je jer omogućava reprezentaciju modulusa u jednom 64-bitnom registru, čime se izbjegava potreba za višestrukom preciznošću i izoluje isključivo vremenski efekat uslovnog grananja. Uz to, ovaj broj je dokazano prost i osigurava validan modulus za RSA operacije. Korištenje 64-bitnog eksponenta (umjesto realnog 2048-bitnog) smanjuje trajanje eksperimenta dok zadržava identičan mehanizam curenja informacija: svaki bit se napada nezavisno, pa se napad praktično linearno skalira s brojem bita. PRNG seed je fiksiran radi reproducibilnosti; kod prihvata seed kao argument (`-seed`), a u svim prikazanim eksperimentima korištena je vrijednost 0xABCDEF1234567890.

3.6 Implementacija Flush+Reload napada

Za razliku od prethodnih eksperimenata koji mjere ukupno trajanje RSA operacije unutar jednog procesa, Flush+Reload napad koristi *dva odvojena procesa*, žrtvu i napadača, koji nikada ne komuniciraju direktno. Jedino što ih povezuje je dijeljeni LLC cache, koji nastaje jer operativni sistem mapira istu dijeljenu biblioteku (`.so`) u oba procesa kroz iste fizičke memorijske stranice (sekcija 2.3.4).

Implementacija je kontrolisani dokaz koncepta koji namjerno izoluje ciljanu funkciju radi jasne demonstracije mehanizma. Međutim, identičan obrazac, uslovno množenje u zasebnoj funkciji, postojao je u stvarnim bibliotekama: GnuPG/libgcrypt je koristio zasebnu `_gcry_mpi_mul()` funkciju za korak množenja, što su Yarom i Falkner [14] iskoristili u originalnom Flush+Reload napadu. Starije verzije OpenSSL-a su imale sličnu separaciju između `bn_sqr i bn_mul_mont`.

3.6.1 Dijeljeni RSA kod

RSA implementacija smještena je u dijeljenu biblioteku `librsa_shared.so`. Ključna dizajnerska odluka je izolovanje uslovnog množenja (koraka koji se izvršava samo za `bit=1`) u zasebnu funkciju:

Program 3.8: Ciljana funkcija za Flush+Reload napad

```
uint64_t __attribute__((noinline, aligned(64)))
rsa_multiply_step(uint64_t result, uint64_t base,
                  uint64_t mod) {
    return mulmod(result, base, mod);
}
```

Dva atributa su neophodna za ispravnost napada:

- `noinline` sprječava kompajler da ugradi tijelo funkcije u pozivajući kod, čime bi nestala zasebna adresa koju napadač može pratiti.
- `aligned(64)` poravnava funkciju na granicu cache linije da ne bi `rsa_multiply_step()` mogla dijeliti cache liniju s okolnim kodom, pa bi `clflush` te linije izbacivao i uzrokovao lažne cache pogotke.

Funkcija `rsa_decrypt_shared()` poziva `rsa_multiply_step()` samo kad je bit eksponenta = 1:

Program 3.9: Uslovno pozivanje ciljane funkcije

```
for (int i = EXP_BITS - 1; i >= 0; i--) {
    result = mulmod(result, result, mod);
    if ((exp >> i) & 1)
        result = rsa_multiply_step(result, base, mod);
}
```

Ovo uslovno pozivanje je upravo ono što napadač iskorištava: kad je `bit = 1`, žrtva pristupa cache liniji funkcije `rsa_multiply_step()` i ostavlja trag u LLC-u; kad je `bit = 0`, ta cache linija ostaje netaknuta.

3.6.2 Žrtva i napadač

Žrtva (`victim_fr`). Simulira RSA server koji u petlji dekriptuje poruke tajnim ključem `0xDEADBEEFCAFEBADE`, pozivajući `rsa_decrypt_shared()` iz dijeljene biblioteke. Između dekriptovanja pauzira $50\ \mu\text{s}$ da stvori jasne granice između operacija. Žrtva se fiksira na CPU 2, koji prema topologiji jezgri pripada drugom fizičkom P-jezgru, izolovanom od CPU 0 koji koristi napadač. Ova izolacija je ključna: ako bi žrtva i napadač dijelili isto fizički jezgro (npr. CPU 0 i CPU 1 kao hiperniti), zajednički predviđač grananja bi naučio obrazac ključa i spekulativno izvršavao `rsa_multiply_step()` čak i za `bit=0`, uzrokujući lažne cache pogotke.

Napadač (`attacker_fr`). Nema nikakav pristup memoriji žrtve niti zna tajni ključ. Jedino što posjeduje je adresa funkcije `rsa_multiply_step()` u dijeljenom `.so` fajlu. Napadač izvodi 100 000 Flush→Čekanje→Reload ciklusa i klasificira svaku reload latenciju:

- **Kratka latencija** (≈ 340 ciklusa): Žrtva je u međuvremenu pozvala funkciju i učitala je u cache $\Rightarrow$ bit = 1.
- **Duga latencija** (≈ 681 ciklusa): Funkcija je ostala u DRAM-u $\Rightarrow$ bit = 0 (ili žrtva nije bila aktivna).

Kritičan implementacijski detalj je pribavljanje adrese ciljane funkcije. Uzimanje adrese operatorom `&` u C-u (npr. `&rsa_multiply_step`) daje adresu PLT (*Procedure Linkage Table*) stuba u napadačevom binarnom fajlu, a ne stvarnu adresu u `.so`. Budući da žrtva poziva stvarnu adresu, `clflush` na PLT stubu ne bi uklonio cache liniju koju žrtva koristi, i napad ne bi radio. Ispravno rješenje je `dlsym()` koji vraća stvarnu adresu u dijeljenom objektu:

Program 3.10: Pribavljanje stvarne adrese ciljane funkcije

```
void *handle = dlopen("./librsa_shared.so",  
                    RTLD_NOLOAD | RTLD_NOW);  
void *target = dlsym(handle, "rsa_multiply_step");
```

3.6.3 Kalibracija praga

Da bi napadač razlikovao cache hit od cache miss-a, potreban je prag latencije. Ovaj prag se određuje eksperimentalno pri pokretanju programa:

1. Mjeri se latencija pristupa nedavno korištenoj adresi (cache hit referenca): ≈ 340 ciklusa.
2. Mjeri se latencija pristupa adresi koja je namjerno izbačena iz cache-a (cache miss referenca): ≈ 681 ciklusa.
3. Prag se postavlja na sredinu: $(340 + 681)/2 \approx 511$ ciklusa.

3.7 Napad rekonstrukcije ključa

3.7.1 Struktura napada i model slijepe rekonstrukcije

Napad je implementiran kao slijepa rekonstrukcija u kojoj napadački kod nikada ne vidi vrijednost tajnog ključa S . Tajni ključ pohranjen je isključivo unutar zasebnog modula kao `static` varijabla, nevidljiva ostatku napadačkog koda. Jedini načini interakcije s tim modulom su:

`oracle_secret_time( $M$ )` koji vraća trajanje eksponencijacije s tajnim ključem za odabranu bazu M .

`oracle_perturbed_time( $M, i$ )` koji vraća trajanje eksponencijacije s istim ključem, ali s obrnutim bitom na poziciji i . Tu izmjenu modul primjenjuje interno, bez da otkriva vrijednost S .

`oracle_reveal_secret()` koji otkriva S jedino za provjeru ispravnosti i poziva se tek nakon završene rekonstrukcije svih bita.

Ovaj model odgovara Kocher-ovom scenariju [2]: napadač šalje odabrane ulaze serveru koji rukuje tajnim ključem i mjeri trajanje odgovora, ali nikada ne dobiva direktan pristup ključu.

3.7.2 Princip bit-po-bit napada

Za svaki bit i od najznačajnijeg ka najmanje značajnom, napad radi sljedeće:

1. Izmjeri prosječno trajanje ekspanencijacije s tajnim ključem: $\bar{T}_S$.
2. Izmjeri prosječno trajanje ekspanencijacije s istim ključem, ali s obrnutim bitom na poziciji i : $\bar{T}_{S'}$.
3. Računa razliku $\Delta_i = \bar{T}_S - \bar{T}_{S'}$.
4. Ako je $\Delta_i > 0$: originalni ključ traje duže, što znači da je bit i uzrokovao dodatno množenje, pa je $S[i] = 1$.
5. Ako je $\Delta_i < 0$: ključ s obrnutim bitom traje duže, pa je $S[i] = 0$.

U praksi, implementacija umjesto prostih prosječnih vrijednosti koristi uparenu t-statistiku opisanu u nastavku, koja je otpornija na šum.

3.7.3 Tehnike robustnosti

Napad implementira pet tehnika za robustnost:

Uparena mjerenja i statistika razlika. Za svaki par mjerenja koristi se ista baza M , čime se eliminišu varijacije koje potiču od trenutnog stanja cache memorije. Umjesto poređenja prosječnih vrijednosti $\bar{T}_S$ i $\bar{T}_{S'}$, za svaki par k računa se razlika $\delta_k = T_S^{(k)} - T_{S'}^{(k)}$, a potom t-statistika:

$$t_i = \frac{\bar{\delta}}{\hat{\sigma}_\delta / \sqrt{K}}$$

Bit se klasificira prema predznaku t_i , a veličina $|t_i|$ mjeri pouzdanost te odluke.

Ponavljjanje mjerenja za nesigurne bite. Ako je $|t_i| < 2,5$ (što odgovara pouzdanosti manjoj od 99%) nakon prvog prolaza s K parova, automatski se provodi drugi prolaz s $2K$ parova samo za taj bit. Jasni biti se ne mjere ponovo i napad troši dodatno vrijeme isključivo tamo gdje je signal nejasan.

Neutralizacija predviđača grananja. Između svakog para mjerenja izvode se tri modularne ekspanencijacije s nasumičnim eksponentima. Svrha je vraćanje CPU predviđača grananja u neutralno stanje: bez ovog koraka, predviđač bi upamtio obrazac prethodnog eksponenta i sistematski ubrzavao prvo mjerenje sljedećeg para, unoseći pristranost u rezultate.

Odbacivanje anomalnih parova. Odbacuju se parovi u kojima je apsolutna razlika $|T_S - T_{S'}|$ veća od 1 000 ciklusa. Stvarna timing razlika iznosi svega ≈ 75 ciklusa, pa veća razlika ukazuje na to da je prekid konteksta pogodio jedno od dva mjerenja u paru, a ne na stvarnu razliku između ekspanenata.

Skraćena sredina. Za prikaz i upis rezultata koristi se skraćena sredina koja odbacuje gornji i donji 5% uzoraka [23], čime se smanjuje utjecaj preostalih rubnih vrijednosti na prikazane brojke. Sama odluka o vrijednosti bita temelji se na t-statistici računatoj iz cijelog skupa parova. Implementacija i eksperimentalni dizajn opisani u ovom poglavlju čine osnovu za analizu rezultata koja slijedi u narednom poglavlju.

Poglavlje 4

Rezultati i analiza

4.1 Eksperiment 1: Detekcija timing leaka

4.1.1 Deskriptivna statistika

Cilj prvog eksperimenta je odgovoriti na osnovno pitanje: da li ranjiva square-and-multiply implementacija zaista ostavlja mjerljiv trag u trajanju operacije koji zavisi od vrijednosti tajnog bita? Eksperiment koristi paired dizajn opisan u sekciji 3.5, s ciljanim bitom na poziciji 16.

Nakon filtriranja rubnih vrijednosti (*outlier*-a), mjerenja koja označavaju prekid od strane operativnog sistema, prikupljeno je ukupno 980 050 validnih mjerenja za ranjivu implementaciju (odbačeno 2,0%) i 980 437 za constant-time implementaciju (odbačeno 2,0%). Tabela 4.1 prikazuje deskriptivnu statistiku za obje implementacije.

Tabela 4.1: Deskriptivna statistika trajanja modularne eksponencijacije (u CPU ciklusima)

Metrika	Ranjiva implementacija			Constant-time		
	bit=0	bit=1	Δ	bit=0	bit=1	Δ
<i>N</i> uzoraka	490 003	490 047	—	490 150	490 287	—
Mean	8816,86	8892,77	+75,90	10689,68	10689,82	+0,14
Medijan	8824,00	8897,00	+73,00	10554,00	10554,00	0,00
Std Dev	561,72	571,81	+10,09	326,16	326,22	+0,06
P25	8406,00	8474,00	+68,00	10530,00	10530,00	0,00
P75	9193,00	9275,00	+82,00	10619,00	10620,00	+1,00

Rezultati za ranjivu implementaciju pokazuju jasnu i konzistentnu razliku između grupa bit=0 i bit=1. Prosječno trajanje operacije kada je tajni bit jednak 1 duže je za 75,90 CPU ciklusa u odnosu na slučaj kada je bit jednak 0. Ova razlika nije prisutna samo u aritmetičkoj sredini, već se proteže kroz sve mjere centralne tendencije: medijan pokazuje razliku od 73 ciklusa, donji kvartil (P25) 68 ciklusa, a gornji kvartil (P75) 82 ciklusa. Konzistentnost razlike na svim percentilima potvrđuje da se radi o pomjeranju cijele distribucije, a ne o slučajnosti koju bi mogle uzrokovati ekstremne vrijednosti.

Fizičko objašnjenje ove razlike je jasno: kada je bit eksponenta jednak 1, ranjiva implementacija izvršava dodatni poziv `mulmod()` (množenje s bazom), što zahtijeva jednu dodatnu 64-bitnu `MULQ` instrukciju i `DIVQ` instrukciju za modulo operaciju. Ove instrukcije zajedno traju upravo oko 75 ciklusa na testiranom Intel i5-1335U procesoru, pri čemu `DIVQ` zauzima većinu tih ciklusa jer je jedna od najskupljih x86 instrukcija (60–90 ciklusa).

Za implementaciju sa konstantnim vremenom razlika u sredini iznosi svega $+0,14$ ciklusa, dok su medijan i P25 potpuno identični za obje grupe. Ovo je očekivano jer se uvijek izvršavaju oba koraka (kvadriranje i množenje) bez obzira na vrijednost bita, pa procesor ne može razlikovati dva slučaja po trajanju.

4.1.2 Statistički testovi

Deskriptivna statistika nagovještava postojanje timing leaka, ali za formalni dokaz potrebni su statistički testovi koji uzimaju u obzir varijabilnost podataka i veličinu uzorka. Tabela 4.2 prikazuje rezultate šest komplementarnih statističkih metrika.

Tabela 4.2: Rezultati statističkih testova

Test	Ranjiva		Constant-time	
	Statistika	p-vrijednost	Statistika	p-vrijednost
Welch's t-test	$t = -66,29$	$< 10^{-50}$	$t = -0,21$	0,836
Paired t-test	$t = -115,43$	$< 10^{-50}$	$t = -0,51$	0,614
Mann-Whitney U	$U = 1,11 \times 10^{11}$	$< 10^{-50}$	$U = 1,20 \times 10^{11}$	0,327
Cohen-ov d	$d = 0,134$		$d \approx 0$	
Bootstrap 95% CI (Δ mean)	[+73,64, +78,13] cik.		[-1,14, +1,42] cik.	

Za ranjivu implementaciju svi testovi jednoglasno odbacuju nultu hipotezu H_0 (da nema razlike u trajanju između grupa). P-vrijednosti su daleko ispod 10^{-50} , što znači da je vjerojatnoća dobivanja ovakve razlike slučajno praktično jednaka nuli. Konkretno:

Paired t-test ($t = -115,43$) je primarni test za ovaj eksperimentalni dizajn jer koristi uparenu strukturu podataka. Budući da su parovi eksponenata identični na svim bitovima osim ciljanog, paired t-test eliminira svu varijabilnost koja dolazi od razlika između eksponenata (npr. različita Hammingova težina) i izoluje isključivo efekat ciljanog bita. Welch-ov t-test je uključen kao konzervativnija alternativa koja ne pretpostavlja parnu strukturu, dok Mann-Whitney U test ne pretpostavlja normalnu distribuciju podataka.

Bootstrap interval povjerenja pruža interpretaciju koja ne zavisi od distribucijskih pretpostavki: sa 95% sigurnosti, prava timing razlika leži između 73,64 i 78,13 ciklusa. Ovaj rezultat je dobiven ponovljenim uzorkovanjem s vraćanjem (10 000 iteracija) iz originalnih podataka.

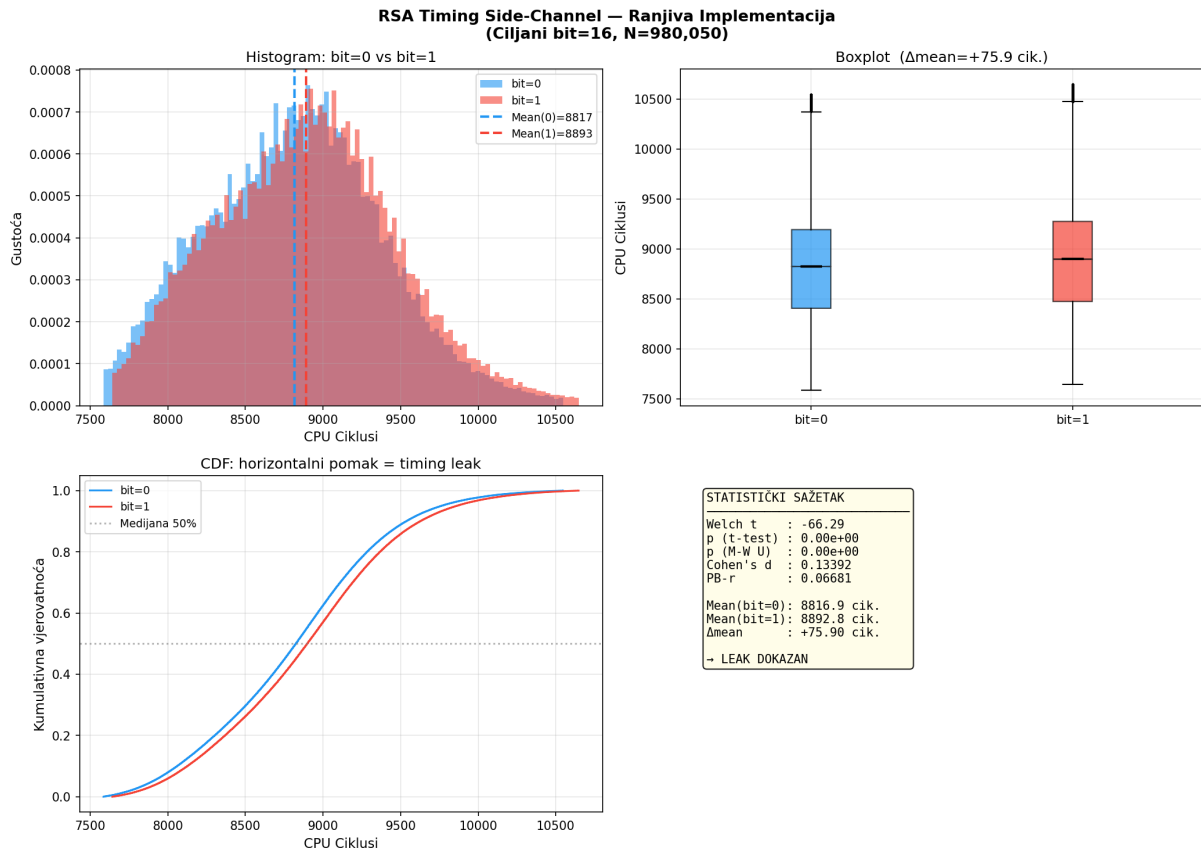
Cohen-ov $d = 0,134$ mjeri veličinu efekta u jedinicama standardne devijacije. Vrijednost $d = 0,134$ znači da je signal (76 ciklusa) oko 7,5 puta manji od šuma ($\sigma \approx 567$ ciklusa). Po njegovoj klasifikaciji, ovo je "mali efekt". Međutim, za timing napad ova klasifikacija nije direktno povezana sa praktičnošću jer se efekt može pojačati usrednjavanjem, kao što demonstrira Eksperiment 2.

Power analiza pokazuje da je minimalan broj uzoraka za detekciju ovog efekta $N_{\min} = 438$ (uz konvencionalne parametre $\alpha = 0,05$ i statističku snagu 0,80). Sa 490 000+ uzoraka po grupi, eksperiment je daleko iznad ovog praga, što garantuje visoku pouzdanost zaključaka.

Za konstantu vremensku implementaciju, nijedan test ne odbacuje H_0 : p-vrijednosti iznose 0,84, 0,61 i 0,33 za tri testa, što su tipične vrijednosti kada nikakva razlika ne postoji. Cohen-

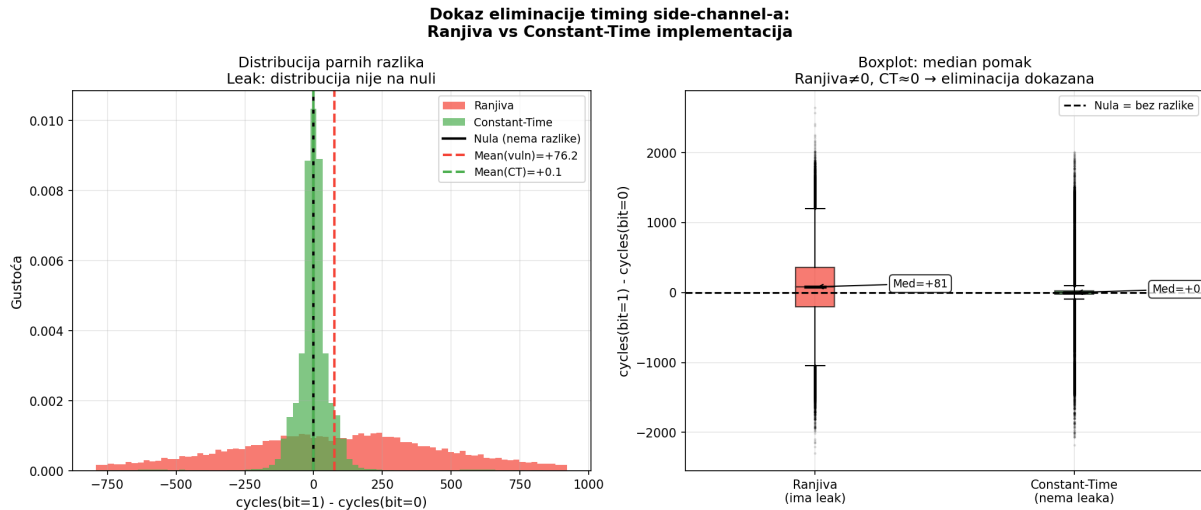
ov $d \approx 0$ i $r \approx 0$ potvrđuju odsustvo efekta. Power analiza pokazuje da bi za detekciju ovako malog “efekta” bilo potrebno $N_{\min} = 45\,141\,981$ uzoraka, što praktično znači da nije detektovan mjerljiv vremenski signal.

4.1.3 Vizualizacija distribucija



Slika 4.1: Distribucija trajanja ranjive implementacije: histogram, boxplot i CDF za grupe bit=0 i bit=1.

Na slici 4.1 histogram prikazuje dvije distribucije koje se gotovo potpuno preklapaju, ali s vidljivim horizontalnim pomakom od oko 75 ciklusa. Preklapanje je očekivano s obzirom na mali Cohen-ov d : šum (varijansa od cache miss-ova, branch mispredictions i preempcija operativnog sistema) je znatno veći od signala. CDF (kumulativna funkcija distribucije) krivulje jasno ilustruju vremensko curenje: za isti postotak uzoraka, distribucija grupe bit=1 konzistentno zahtijeva veći broj ciklusa. Paralelnost CDF krivulja potvrđuje da je pomak uniforman preko cijele distribucije, a ne lokaliziran na određenom dijelu.



Slika 4.2: Distribucija parnih razlika trajanja za ranjivu (crvena) i konstantnu vremensku (zelenu) implementaciju.

Slika 4.2 direktno prikazuje distribuciju razlika unutar svakoga para. Za ranjivu implementaciju, centar distribucije je jasno pomaknut udesno od nule (centar na +76 ciklusa), dok za constant-time implementaciju distribucija leži precizno na nuli. Ovo je najintuitivniji vizualni dokaz da implementacija s konstantnim vremenom uspješno eliminiše curenje.

4.1.4 Validacija eksperimenta

Tabela 4.3 sumira sedam unaprijed definisanih validacionih kriterija koji zajedno potvrđuju ispravnost eksperimentalnog postupka.

Tabela 4.3: Validacija eksperimenta: svi kriteriji ispunjeni

Kriterij	Uslov	Rezultat
Vremensko curenje postoji	$p_{MW} < 10^{-10}$	$< 10^{-50}$ ✓
T-test značajan	$p < 0,01$	$< 10^{-50}$ ✓
Efekt mjerljiv	$ d > 0,01$	$d = 0,134$ ✓
Napad uspješan	tačnost $> 70\%$	100% za $K = 2000$ ✓
CT eliminira leak	$p_{MW} > 0,05$	$p = 0,327$ ✓
CT bez efekta	$ d < 0,05$	$d \approx 0,000$ ✓
CT napad neuspješan	tačnost $< 60\%$	$\approx 50\%$ ✓

Ispunjenje svih sedam kriterija potvrđuje da eksperimentalni postupak ispravno detektuje vremensko curenje kada postoji, i ispravno ne detektuje ništa kada ga nema. Ovo isključuje mogućnost da su rezultati posljedica metodološke greške.

4.2 Eksperiment 2: Napad sa usrednjavanjem

4.2.1 Metodologija

Eksperiment 1 dokazuje da vremensko curenje postoji, ali postavlja se pitanje: može li napadač praktično iskoristiti signal od samo 76 ciklusa u šumu od 567 ciklusa za pogađanje vrijednosti

tajnog bita? Eksperiment odgovara na ovo pitanje tako što demonstrira da usrednjavanje više mjerenja ignoriše šum i izvlači signal.

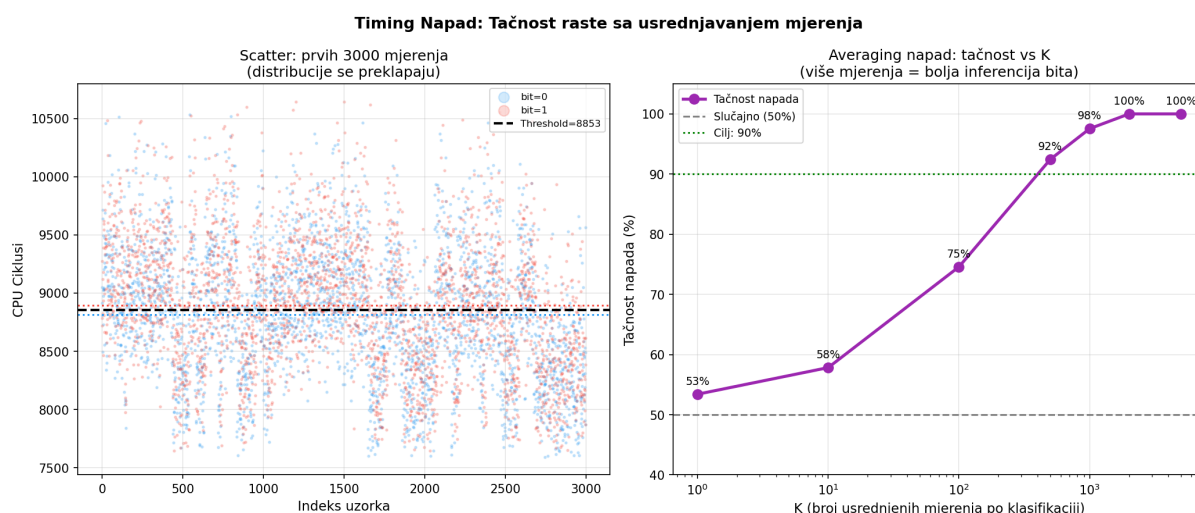
Za svaki nivo usrednjavanja K , napadač izvodi sljedeći postupak: (1) procjenjuje optimalni prag iz prve polovice podataka, (2) formira K nasumičnih mjerenja iz testnog skupa i računa njihovu sredinu, te (3) klasificira usrednjeni rezultat pravilom praga: ako je sredina iznad praga, proglašava bit=1, inače bit=0.

Teorijski minimalan K za postizanje 95% tačnosti izvodi se iz odnosa signala i šuma. Za $\text{SNR} = d = 0,134$ i faktor pouzdanosti $r = 2$ (koji odgovara 95% jednosmjernoj pouzdanosti, jednačina 2.6 daje $K_{\min} = (r/\text{SNR})^2 = (2/0,134)^2 \approx 223$. Intuicija iza ove formule je da standardna devijacija sredine od K mjerenja je $\sigma/\sqrt{K}$, pa sa dovoljno velikim K sredina postaje dovoljno precizna da signal od 76 ciklusa bude jasno odvojen od šuma.

4.2.2 Rezultati

Tabela 4.4: Tačnost timing napada sa usrednjavanjem u funkciji broja usrednjenih mjerenja K

K	Tačnost (ranjiva)	95% CI	Tačnost (CT)
1	53,4%	[51,9%, 54,9%]	50,2%
10	57,8%	[56,3%, 59,3%]	50,0%
100	74,6%	[73,2%, 75,9%]	50,9%
500	92,4%	[90,6%, 93,9%]	50,9%
1000	97,6%	[95,8%, 98,6%]	52,2%
2000	100,0%	[98,4%, 100,0%]	43,9%
5000	100,0%	[96,2%, 100,0%]	51,0%



Slika 4.3: Dijagram raspršenja (*scatter plot*) mjerenja (lijevo) i tačnost u funkciji K (desno).

Slika 4.3 prikazuje rasipanje pojedinačnih mjerenja (lijevo) i rast tačnosti s povećanjem K (desno). Rezultati potvrđuju teorijsko predviđanje. Sa jednim mjerenjem ($K = 1$), napadač pogađa tek 53,4%, jedva iznad slučajnog pogađanja od 50%, jer je signal zakopan u šumu. Sa $K = 100$, tačnost raste na 74,6%, što je blizu teorijskog $K_{\min} = 223$ za 95%. Sa $K = 500$ napadač postiže 92,4%, a sa $K = 2000$ dostiže savršenih 100%.

Fizičko objašnjenje: za $K = 2000$, standardna devijacija sredine iznosi $\sigma_{2000} = 567/\sqrt{2000} \approx 12,7$ ciklusa. Signal od 76 ciklusa je sada šest puta veći od šuma sredine, što čini pogrešnu klasifikaciju praktično nemogućom.

Za implementaciju s konstantnim vremenom tačnost napada ostaje u opsegu 44–52% za sve vrijednosti K , što je konzistentno s odsustvom vremenskog curenja. Vrijednost od 43,9% za $K = 2000$ je statističko odstupanje; 95% interval povjerenja za slučajno pogađanje s 200–500 test blokova obuhvata ovaj raspon.

4.2.3 Osjetljivost na veličinu uzorka

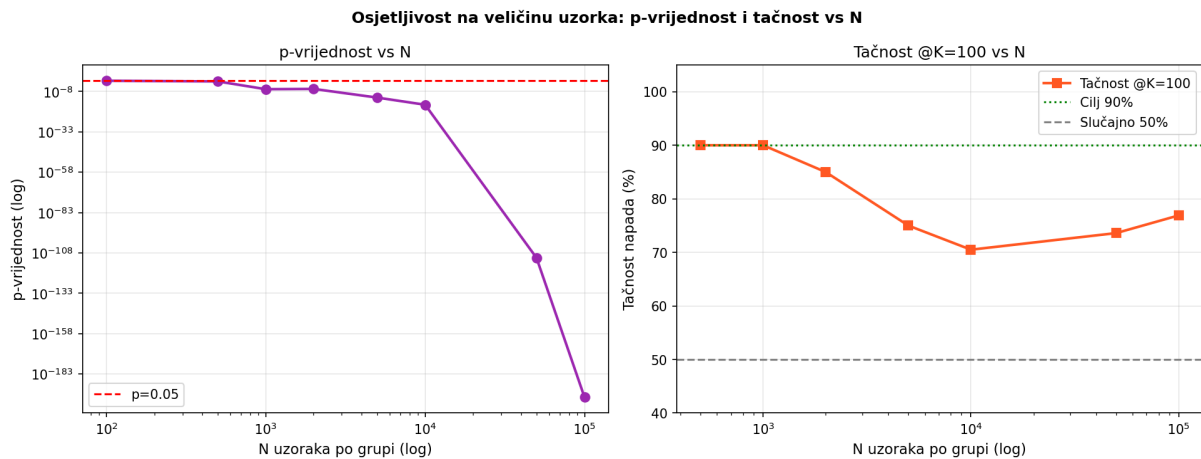
Tabela 4.5 prikazuje kako se statistička značajnost i tačnost napada mijenjaju kada se koristi samo podskup ukupnih podataka. Ova analiza odgovara na pitanje: koliko mjerenja napadač minimalno treba prikupiti?

Tabela 4.5: Osjetljivost na veličinu uzorka: p-vrijednost i tačnost napada

N uzoraka	p-vrijednost	Značajno?	Tačnost ($K = 100$)
100	4,53e-02	Da	N/A
500	1,76e-02	Da	90,0%
1 000	2,65e-07	Da	90,0%
2 000	3,85e-07	Da	85,0%
5 000	1,68e-12	Da	75,0%
10 000	6,14e-17	Da	70,5%
50 000	7,81e-112	Da	73,6%
100 000	3,50e-198	Da	76,9%

Za $N = 100$, usrednjavanje sa $K = 100$ nije moguće jer ukupan broj uzoraka nije dovoljan za formiranje nezavisnih blokova, pa je tačnost označena kao N/A. Najmanji N za postizanje statističke značajnosti ($p < 0,05$) iznosi $N = 100$ ($p = 4,53 \times 10^{-2}$). Čak i sa samo 100 uzoraka timing leak je na granici statističke vidljivosti, uprkos tome što power analiza za 80% snagu predviđa $N_{\min} = 438$. Ovo sugerise da je stvarna veličina efekta u ovom poduzorku bila nešto veća od ukupnog prosjeka.

Naizgled paradoksalan pad tačnosti napada za veće N (npr. 90% za $N = 500$ naspram 70,5% za $N = 10000$ uz $K = 100$) objašnjava se nestabilnošću procjene pri malom broju test blokova. Za $N = 500$ i $K = 100$ postoji samo 5 nezavisnih blokova (svaki od 100 mjerenja), pa jedan ili dva sretna bloka značajno utječu na procijenjenu tačnost. Za veće N , broj blokova raste i procjena konvergira ka stvarnoj tačnosti od približno 75% za $K = 100$, što odgovara teorijskom predviđanju za $\text{SNR} = 0,134$ i K daleko ispod $K_{\min} = 223$.



Slika 4.4: Osjetljivost napada na veličinu uzorka: tačnost u funkciji N za $K = 100$.

4.3 Eksperiment 3: Slijepa rekonstrukcija 64-bitnog privatnog ključa

4.3.1 Metodologija

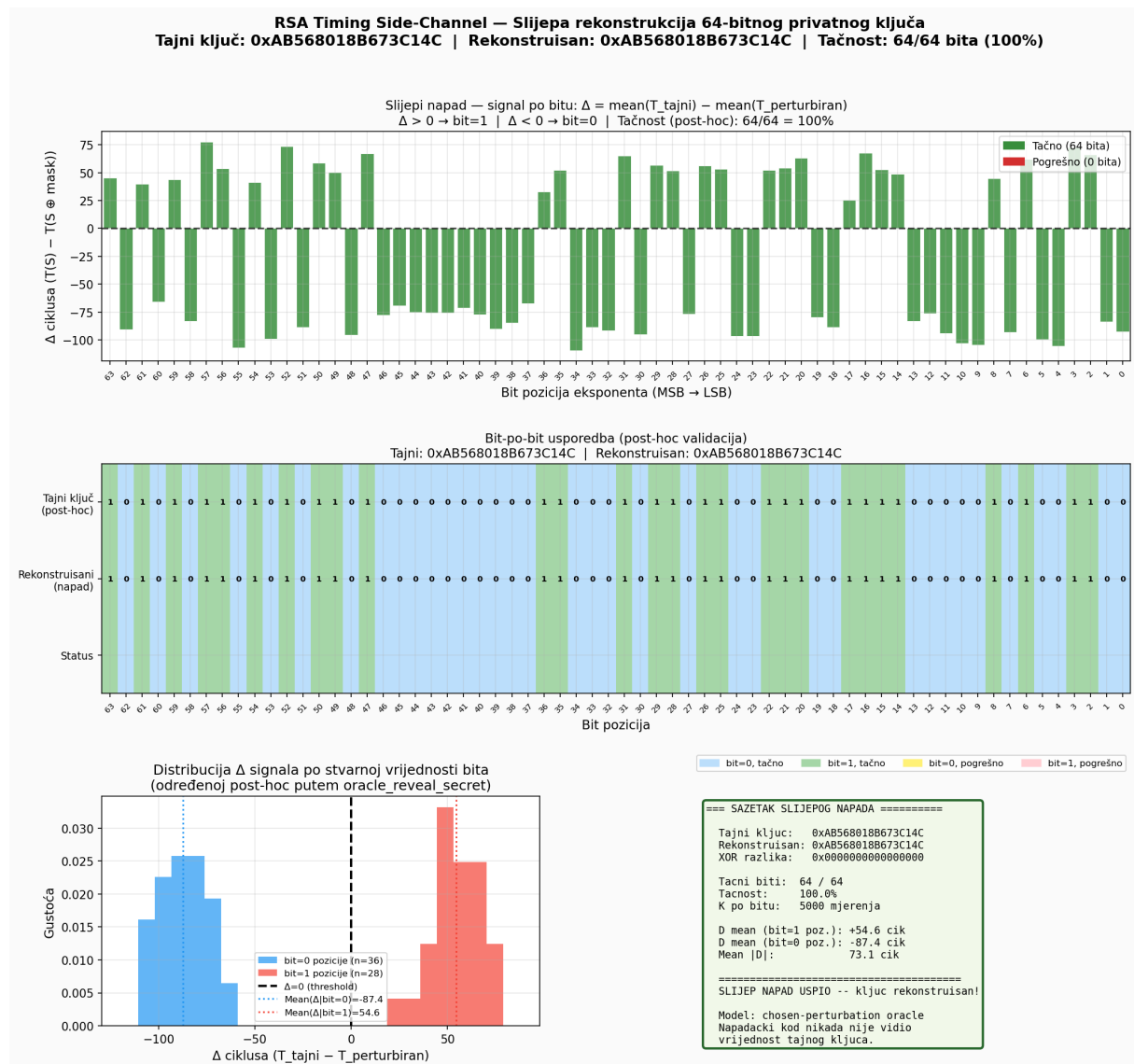
Eksperimenti 1 i 2 pokazuju da napadač može pouzdano odrediti vrijednost jednog bita. Eksperiment 3 primjenjuje isti princip na svih 64 bita privatnog ključa i rekonstruiše cijeli ključ, uz jedno ključno obilježje: napadački kod nikada ne vidi tajni ključ.

Tajni ključ generisan je nasumično iz `/dev/urandom` i pohranjen unutar zasebnog modula kome napadač nema direktan pristup (sekcija 3.7). Za svaki bit napadač pita taj modul koliko dugo traje ekspanencijacija s originalnim ključem i koliko bi trajala da je samo taj jedan bit obrnut. Iz razlike tih trajanja zaključuje koja je bila izvorna vrijednost bita. Ovaj postupak se ponavlja za sva 64 bita, sa $K = 5000$ mjerenja po bitu radi pouzdanosti. Tek na samom kraju, radi provjere tačnosti, modul otkriva tajni ključ.

4.3.2 Rezultati

Tabela 4.6: Rezultati slijepe rekonstrukcije 64-bitnog ključa

Metrika	Vrijednost
Tajni ključ (nasumičan, <code>/dev/urandom</code>)	0xAB568018B673C14C
Rekonstruisani ključ	0xAB568018B673C14C
XOR razlika	0x0000000000000000
Tačni biti	64 / 64 (100,0%)
K po bitu	5 000
Mean $ \Delta $	73,1 ciklusa
Mean Δ (bit=1 pozicije)	+54,6 ciklusa
Mean Δ (bit=0 pozicije)	-87,4 ciklusa



Slika 4.5: Gornji panel: timing razlika Δ za svaki od 64 bita (pozitivna vrijednost = bit=1, negativna = bit=0). Srednji panel: toplotna mapa rekonstruisanog i stvarnog ključa. Donji panel: distribucija svih Δ vrijednosti

Kao što se vidi na slici 4.5, svih 64 bita nasumičnog tajnog ključa su tačno rekonstruisani. Timing razlika Δ je konzistentna i jasno odvojena po predznaku na svim pozicijama: pozicije gdje je tajni bit 1 daju pozitivan Δ (prosjeak +54,6 ciklusa), a pozicije gdje je bit 0 daju negativan Δ (prosjeak -87,4 ciklusa). Predznak Δ direktno otkriva vrijednost bita jer obrtanje bita iz 1 u 0 uklanja jedno množenje i skraćuje trajanje, dok obrtanje iz 0 u 1 dodaje množenje i produžava ga.

Veličina signala varira od pozicije do pozicije: najslabija razlika bila je 25,3 ciklusa (bit 17), a najjača 109,2 ciklusa (bit 34). Ova varijacija je normalna i odgovara rezultatima Eksperimenta 4. Ni za jedan bit nisu bila potrebna ponovna mjerenja: najmanji $|t|$ -statistik iznosio je 8,3, daleko iznad praga od 2,5 koji bi aktivirao drugi prolaz.

Rekonstrukcija je potpuno zasnovana na timing mjerenjima i napadački kod u nijednom trenutku nije pristupio vrijednosti tajnog ključa. Ovo odgovara realnom napadu: server drži privatni ključ, a napadač ga može jedino posredno "posmatrati" kroz razlike u trajanju odgovora.

4.4 Eksperiment 4: Analiza pozicije bita

4.4.1 Motivacija i metodologija

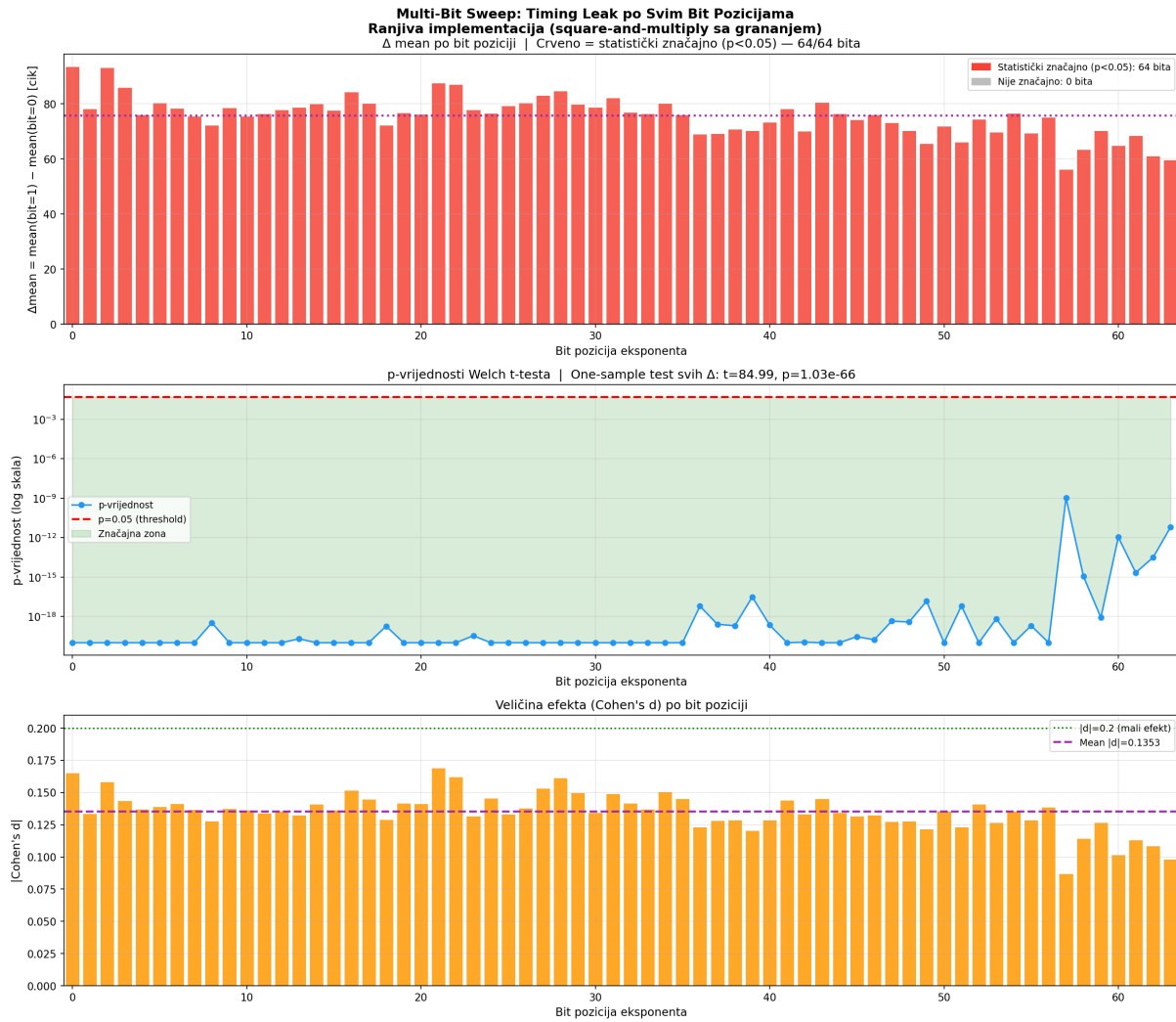
Eksperimenti 1 i 2 ciljaju fiksiranu poziciju bita (bit 16), a Eksperiment 3 primjenjuje napad na svih 64 bita. Prirodno pitanje je: da li je vremensko curenje sistemska osobina koja važi za svaku poziciju, ili postoje pozicije gdje je signal slabiji? Odgovor na ovo pitanje je ključan jer napad rekonstrukcije ključa pretpostavlja da svaki bit nezavisno daje mjerljiv signal.

Eksperiment testira svaku od 64 pozicija bita nezavisno, sa 10 000 uzoraka po poziciji, i za svaku izračunava Welch-ov t-test i Cohen-ov d .

4.4.2 Rezultati

Tabela 4.7: Multi-bit sweep: ukupni rezultati

Metrika	Vrijednost
Statistički značajnih pozicija	64 / 64 (100%)
Mean Δ po svim bitovima	+75,67 ciklusa
Std Δ	7,07 ciklusa
One-sample t-test (H_0 : mean $\Delta = 0$)	$t = 84,993$, $p = 1,027 \times 10^{-66}$



Slika 4.6: Timing razlika Δ po poziciji bita eksponenta sa 95% intervalima povjerenja.

Svih 64 pozicija bita pokazuju statistički značajnu vremensku razliku ($p < 10^{-9}$ za svaku poziciju pojedinačno). Prosječna razlika Δ kreće se od +56,22 ciklusa (bit 57) do +93,49 ciklusa (bit 0), sa ukupnom standardnom devijacijom od 7,07 ciklusa između pozicija.

Varijacija Δ po poziciji (od 56 do 93 ciklusa) nije sistematski trend nego kontekstualni šum: svaka iteracija petlje zatekne različito stanje branch prediktora i cache memorije ovisno o tome koji su se bitovi izvršili ranije, što uvodi slučajnu varijabilnost u izmjerenu razliku. Minimalna vrijednost na bitu 57 i maksimalna na bitu 0 su primjer te slučajnosti i ne postoji krivulja koja pada ili raste s pozicijom. Za realni 2048-bitni ključ slika bi izgledala isto: oko 2048 tačaka grupisanih oko iste srednje vrijednosti (75 ciklusa) s jednakom slučajnom rasutošću. Standardna devijacija od samo 7,07 ciklusa (naspram srednje vrijednosti od 75,67) potvrđuje visoku konzistentnost signala bez obzira na poziciju.

One-sample t-test svih 64 delta vrijednosti daje $t = 84,993$ ($p = 1,027 \times 10^{-66}$), čime se definitivno odbacuje hipoteza da je prosječna razlika jednaka nuli. Ovaj rezultat potvrđuje da vremensko curenje nije nastalo zbog jedne bit pozicije, već je osobina ranjive square-and-multiply implementacije: svaki bit eksponenta nezavisno doprinosi mjerljivoj vremenskoj razlici.

4.5 Eksperiment 5: Utjecaj kompajlerskih optimizacija

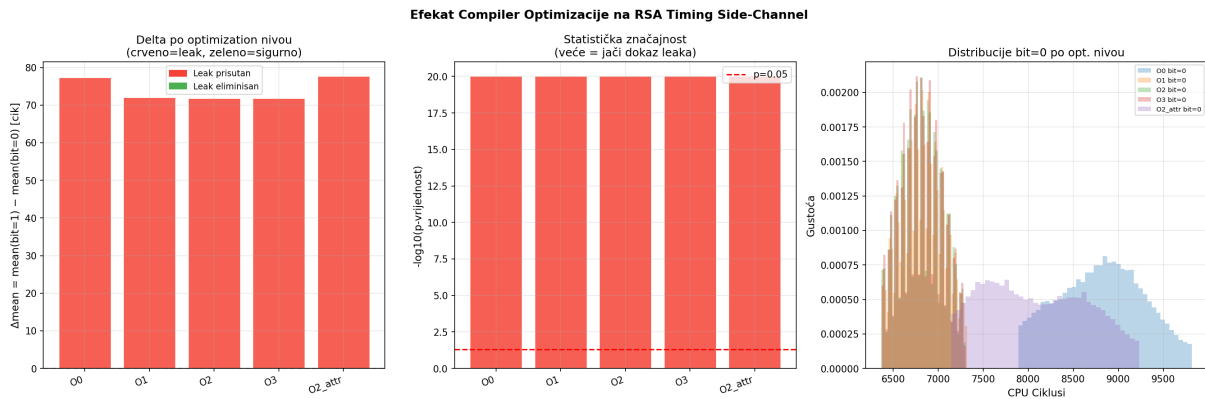
4.5.1 Motivacija

U sekciji 3.4.2 navedeno je da ranjiva implementacija koristi `optimize("O0")` atribut koji isključuje optimizacije. Postavlja se praktično pitanje: može li kompajler s višim nivoom optimizacije automatski eliminisati timing leak, na primjer transformacijom uslovnog grananja (`if`) u instrukciju uslovnog pomaka (`CMOV`) koja se izvršava u konstantnom vremenu? Ako bi to bio slučaj, programeri bi se mogli osloniti na kompajler umjesto na eksplicitne kontramjere. Ovaj eksperiment testira pet varijanti iste ranjive implementacije kompajlirane s različitim optimizacijskim nivoima i provjerava generisani asemblerski kod.

4.5.2 Rezultati

Tabela 4.8: Timing leak po nivoima kompajlerske optimizacije

Varijanta	Opt.	Atribut	Δ mean	p-vrijednost	Cohen-ov d	Assembly
O0	-O0	Ne	+77,29	$< 10^{-50}$	0,136	branch
O1	-O1	Ne	+72,01	$< 10^{-50}$	0,267	branch
O2	-O2	Ne	+71,73	$< 10^{-50}$	0,267	branch
O3	-O3	Ne	+71,83	$< 10^{-50}$	0,267	branch
O2+attr	-O2	Da	+77,62	$< 10^{-50}$	0,120	branch



Slika 4.7: Utjecaj kompajlerskih optimizacija: prosječna timing razlika Δ (u CPU ciklusima) i Cohen-ov d za pet varijanti ranjive implementacije.

Nijedan nivo optimizacije ne eliminiše vremensko curenje. Pregled asemblerskog koda za sve varijante (vidjeti Prilog A.7) potvrđuje da GCC ne transformiše grananje u `CMOV` instrukciju ni za jedan nivo optimizacije, a razlog je tehničke prirode: tijelo uslovnog bloka sadrži poziv funkcije (`mulmod`), a `CMOV` može zamijeniti samo jednostavne dodjele vrijednosti, ne pozive funkcija. Pri `-O0` i `O2+attr` kompajler generiše je (obrazac `shrx/and/test/je`), a pri `-O1--O3` generiše `jae` (obrazac `bt/jae`); u oba slučaja radi se o uslovnom skoku koji mijenja putanju izvršavanja ovisno o vrijednosti tajnog bita.

Zanimljiv nalaz je da optimizacije `-O1` do `-O3` zapravo pojačavaju vremensku razliku Cohen-ov d raste sa 0,136 (`O0`) na 0,267 (`O1–O3`), što je gotovo dvostruko povećanje. Fizičko objašnjenje: optimizirani kod drži varijable u registrima umjesto na steku i generiše kompaktniji

pristup memoriji, što smanjuje ukupnu varijansu (σ). Budući da je signal ($\Delta \approx 72$ ciklusa) gotovo isti, manji šum daje veći odnos signala i šuma (SNR).

Varijanta "O2+attr" koristi -02 nivo za cijeli program, ali ciljanu funkciju označava atributom `optimize("O0")`. Ova varijanta ima nešto veći Δ (+77,62 naspram +71,73) ali manji Cohen-ov d (0,120) jer neoptimizirana funkcija ima veću varijansu.

Ovaj eksperiment demonstrira da se programeri ne mogu osloniti na kompajlerske optimizacije kao sigurnosnu mjeru. Jedino eksplicitne vremenski konstantne implementacije, poput one opisane u sekciji 3.4.3, garantuju odsustvo vremenskog curenja neovisno o kompajleru i nivou optimizacije.

4.6 Eksperiment 6: Evaluacija RSA blindinga

4.6.1 Motivacija

RSA blinding je kontramjera koju je OpenSSL implementirao nakon Brumley-Boneh napada [4]. Principi standardnog RSA blindinga opisani su u sekciji 2.3.6. U ovom eksperimentu korištena je pojednostavljena varijanta blindinga:

$$\text{blinded_base} = M \cdot r \bmod n \quad (4.1)$$

$$s' = \text{blinded_base}^d \bmod n \quad (4.2)$$

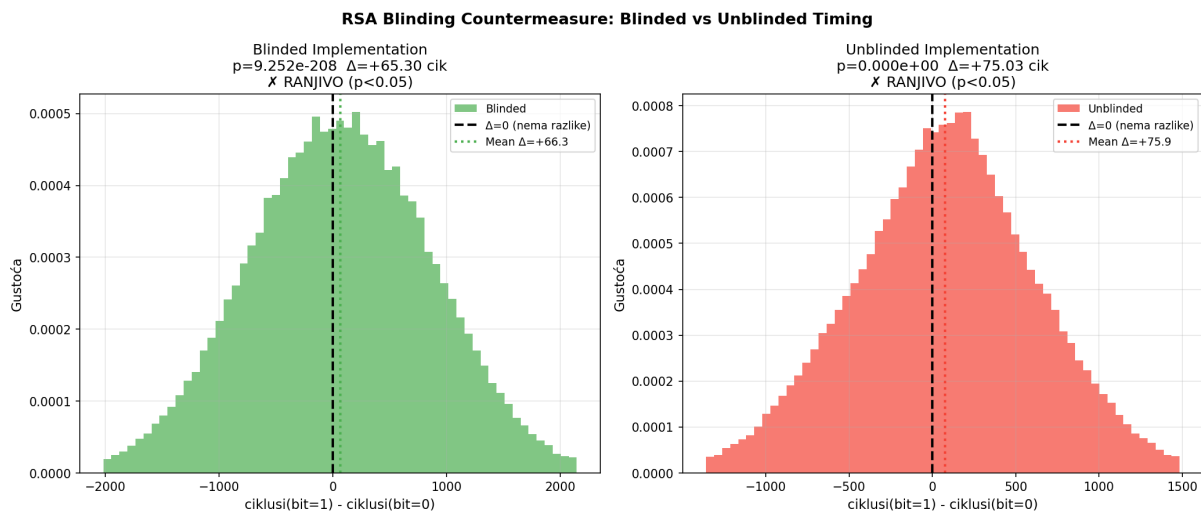
Unblinding u ovoj varijanti nije matematički egzaktn jer $(M \cdot r)^d \neq M^d \cdot r$, ali za analizu timing signala ispravnost krajnjeg rezultata nije relevantna. Ovaj pristup je validan jer mjerimo isključivo trajanje operacije ekspanencijacije, ne njen izlaz, a nasumičan odabir baze vjerno replicira efekat blindinga na interne međuvrijednosti tokom ekspanencijacije. Konkretno, promjenom baze mijenjaju se svi međurezultati tokom ekspanencijacije, što je suštinski jednak efekt onome što standardni blinding postiže na vremensku distribuciju iako konačni rezultat nije matematički egzaktn dekript.

Eksperiment poredi blinded i unblinded varijantu sa oko 196 000 uzoraka po varijanti.

4.6.2 Rezultati

Tabela 4.9: Evaluacija RSA blindinga kao kontramjere

Varijanta	N	Δ	t-stat	p-vrijednost	Cohen-ov d	Štiti?
Blinded	195 999	+65,30	-30,78	$9,3 \times 10^{-208}$	0,098	Ne
Unblinded	196 035	+75,03	-39,15	$< 10^{-50}$	0,125	Ne



Slika 4.8: Evaluacija RSA blindinga kao kontramjere: distribucija parnih timing razlika za blinded (zelena) i unblinded (crvena) varijantu

Blinding smanjuje timing razliku sa +75,03 na +65,30 ciklusa (redukcija od približno 13%) i Cohen-ov d sa 0,125 na 0,098. Međutim, signal i dalje ostaje statistički visoko značajan ($p = 9,3 \times 10^{-208}$), što znači da blinding ne eliminiše vremensko curenje.

Smanjenje signala od 13% ima dva moguća objašnjenja. Prvo, instrukcija `MULQ` na testiranom procesoru možda ima male operand-zavisne timing varijacije koje blinding djelomično maskira promjenom ulaznih operandata. Drugo, blinded varijanta ima duže ukupno trajanje ($\approx 11\,740$ naspram $\approx 8\,770$ ciklusa za unblinded) jer koristi veće brojeve, što povećava apsolutnu varijansu i time razvodnjava relativnu veličinu signala. Razdvajanje ovih dviju hipoteza zahtijevalo bi izoliranu analizu latencije `MULQ`.

4.6.3 Analiza: zašto blinding ne eliminiše leak

Ovaj rezultat nije posljedica implementacijske greške, već fundamentalna posljedica arhitekture napada. U ovoj 64-bitnoj implementaciji, vremensko curenje dolazi od grananja u square-and-multiply algoritmu: za `bit=1` izvršava se dodatni poziv `mulmod()` (grananje), dok se za `bit=0` taj poziv preskače.

Blinding mijenja bazu (ulaz u množenje), ali ne mijenja eksponent d . Obrazac grananja `if ((exp >> i) & 1)` ostaje identičan bez obzira na to koja je baza korištena. Dodatno, 64-bitna `MULQ` instrukcija na x86 izvršava množenje u konstantnom vremenu za sve operande, pa blinding ne može maskirati operand-zavisne varijacije jer ih na ovoj širini operanda praktično nema.

U realnom RSA s big-number aritmetikom (biblioteke poput GMP ili OpenSSL), situacija je suštinski drugačija. Višeprecizno množenje velikih brojeva (npr. 2048-bitnih) nije constant-time jer trajanje zavisi od efektivne veličine operandata i broja propagacija carry bita. U tom scenariju blinding randomizira međuvrijednosti i maskira operand-zavisni timing signal, čineći ga efikasnom kontramjerom [4].

Ovaj eksperiment demonstrira važnu razliku između dvije različite vrste vremenskog curenja:

Branch-zavisni leak (korišten u ovim eksperimentima): Trajanje zavisi od kontrolnog toka, tj. od toga koji se put kroz kod izvršava. Blinding ne pomaže jer ne mijenja kontrolni tok.

Operand-zavisni leak (prisutan u realnom RSA s aritmetikom velikih brojeva): Trajanje zavisi od numeričke veličine operanada. Blinding pomaže jer nasumično bira operande čime onemogućuje napadaču da poveže trajanje s poznatim ulazom.

4.7 Eksperiment 7: Flush+Reload cache napad

4.7.1 Motivacija

Eksperimenti 1–6 demonstriraju timing side-channel napade koji mjere ukupno trajanje kriptografske operacije. Međutim, postoji fundamentalno drugačija klasa napada koja umjesto ukupnog trajanja prati prisutnost specifičnih funkcija u CPU cache memoriji i njihovo vrijeme pristupa. Flush+Reload napad [14] koristi fizički dijeljenu Last Level Cache (LLC) između procesa i može detektovati izvršavanje pojedinačnih funkcija iz dijeljene biblioteke bez mjerenja ukupnog trajanja.

Ova klasa napada ima nekoliko praktičnih prednosti pred timing napadom. Napad funkcionise čak i kada je žrtva na drugom CPU jezgri (bez dijeljenog L1/L2 cache-a), radi u prisustvu jačeg šuma (mrežna aktivnost, virtualizacija) jer signal ne zavisi od ukupnog trajanja operacije, i cilja specifičnu funkciju umjesto globalnog vremenskog signala.

Razlika od 341,5 ciklusa između cache hit-a i miss-a daleko je jača od vremenskog signala iz prethodnih eksperimenata (76 ciklusa), što čini klasifikaciju trivijalno lakom bez statističkog usrednjavanja.

4.7.2 Princip Flush+Reload napada

Princip Flush+Reload napada opisan je u sekciji 2.3.4: napadač u ciklusu izbacuje cache liniju ciljane funkcije (`clflush`), čeka da žrtva potencijalno izvrši operaciju, a potom mjeri reload latenciju. Kratka latencija (≈ 340 ciklusa) ukazuje na cache hit i zaključuje da je bit eksponenta jednak 1; duga latencija (≈ 681 ciklusa) ukazuje na cache miss i zaključuje da je bit jednak 0. Na x86 arhitekturi, svi procesi koji mapiraju isti .so fajl dobijaju iste fizičke okvire stranica, što je preduvjet za dijeljenu LLC cache memoriju.

4.7.3 Implementacija

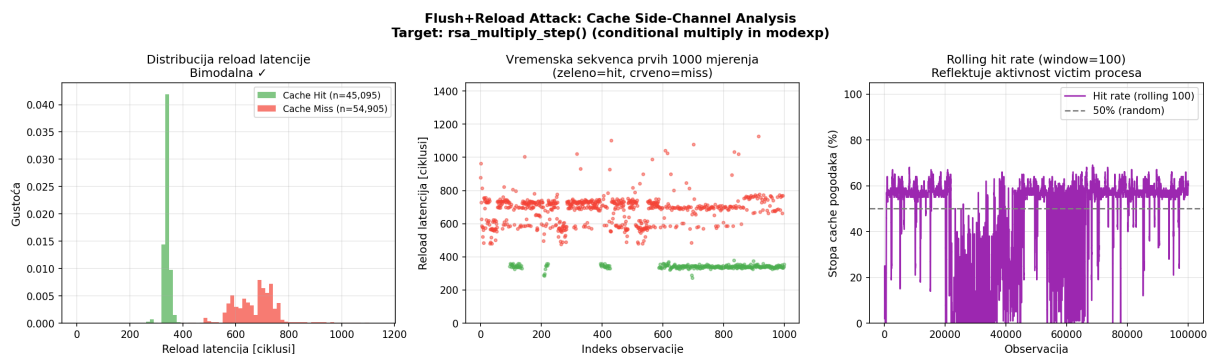
Implementacijski detalji (dijeljeni RSA kod, pribavljanje adrese putem `dl_sym()`, CPU pinning žrtve na zasebni fizički core i kalibracija praga) opisani su u sekciji 3.6. Žrtva između dekriptacija pauzira $50\mu s$ ¹, što daje jasne granice između operacija i odgovara tipičnom primjeru kriptografskog servera s umjerenim opterećenjem.

¹Pri trajanju jedne dekriptacije od $\approx 4,2\mu s$, aktivna frakcija iznosi $4,2/(4,2 + 50) \approx 7,7\%$ ukupnog vremena.

4.7.4 Rezultati

Tabela 4.10: Rezultati Flush+Reload napada na `rsa_multiply_step()`

Metrika	Vrijednost
Broj observacija	100 000
Cache hitovi (bit=1)	45 095 (45,1%)
Cache miss-ovi (bit=0)	54 905 (54,9%)
Mean reload latencija (hit)	339,9 ciklusa
Mean reload latencija (miss)	681,4 ciklusa
Razlika (miss – hit)	341,5 ciklusa



Slika 4.9: Histogram reload latencija: bimodalna distribucija, vremenska sekvenca mjerenja i rolling hit rate

Slika 4.9 prikazuje bimodalnu distribuciju reload latencija sa dva jasno odvojena pika. Cache hit (žrtva je pozvala `rsa_multiply_step()`, što znači da je bit eksponenta jednak 1) daje prosječnu latenciju od 339,9 ciklusa, što odgovara latenciji pristupa LLC-u. Cache miss (funkcija nije bila pozvana) daje 681,4 ciklusa, što odgovara latenciji pristupa glavnoj memoriji (DRAM). Separacija od 341,5 ciklusa između cache hit-a i miss-a približno je 4,5 puta veća od timing signala iz prethodnih eksperimenata.

Udio cache hitova od 45,1% je veći od naivne procjene zasnovane na aktivnoj frakciji i udjelu jediničnih bita ($7,7\% \times 72\% \approx 5,5\%$). Ova razlika objašnjava se perzistencijom podataka u LLC-u: kada žrtva pozove `rsa_multiply_step()`, odgovarajuća cache linija ostaje u LLC-u znatno duže od jednog intervala. Budući da se unutar jedne dekriptacije funkcija poziva za svaki od ≈ 34 bita koji su jednaki 1, cache linija je prisutna u LLC-u tokom značajnog dijela aktivnog perioda žrtve, i napadač može detektovati hit čak i ako je konkretni poziv završio ranije.

4.7.5 Poređenje s timing napadom

Flush+Reload i timing napad ciljaju istu osnovnu ranjivost (uslovno grananje u square-and-multiply), ali koriste suštinski različite mehanizme detekcije:

Tabela 4.11: Poređenje timing napada i Flush+Reload napada

Dimenzija	Timing napad (Eksperimenti 1–4)	Flush+Reload (Eksperiment 7)
Signal	≈ 76 ciklusa	≈ 342 ciklusa
Granularnost	Cijela operacija	Pojedinačna funkcija
Šum	Visok (OS, cache, TLB)	Nizak (bimodalna distribucija)
Mjerenja/bit	$K \geq 2000$ za 100%	1 mjerenje (threshold)
Dijeljeni resursi	Ništa	Dijeljeni LLC
Constant-time zaštita	Eliminiše napad	Eliminiše napad ²

Najznačajnija razlika je u potrebnom broju mjerenja. Timing napad zahtijeva usrednjavanje $K = 2000$ mjerenja po bitu za postizanje 100% tačnosti, dok Flush+Reload može klasificirati bit iz jednog jedinog mjerenja zahvaljujući daleko većem signalu (341 naspram 76 ciklusa). S druge strane, Flush+Reload zahtijeva da napadač i žrtva dijele istu fizičku memoriju (isti .so fajl), što ograničava primjenjivost na scenarije s dijeljenim bibliotekama.

Ova Flush+Reload implementacija je kontrolisani dokaz koncepta s namjerno izolovanom ciljnom funkcijom. U stvarnim kriptografskim bibliotekama, razlika između kvadriranja i množenja u funkcijama postojala je prirodno. Upravo takve prirodne separacije su motivisale Yarom i Falkner [14] da demonstriraju ovaj napad i time potaknu prelazak na konstantne vremenske implementacije u modernim kriptografskim bibliotekama.

4.8 Diskusija

4.8.1 Interpretacija Cohen-ovog $d = 0,134$

Mali Cohen-ov d ne umanjuje značaj nalaza. Ova vrijednost kvantificira fizičku veličinu efekta: signal od 76 ciklusa u šumu od $\sigma \approx 567$ ciklusa daje odnos signala i šuma (SNR) od 0,134 po pojedinačnom uzorku. Ovakav odnos je tipičan za timing napade na modernom procesoru, gdje su izvori šuma znatni (cache miss-ovi, pogrešna predviđanja grananja, preempcije operativnog sistema).

Ono što čini ovaj signal iskoristivim za napad je njegova konzistentnost i determinističnost. Signal se pojavljuje identično na medijani, P25 i P75 (Eksperiment 1), na svih 64 pozicija bita (Eksperiment 4), i pri različitim nivoima optimizacije (Eksperiment 5). Budući da je signal konzistentan a šum slučajan, usrednjavanje ga pouzdano "izvlači": $K = 2000$ uzoraka je dovoljno za 100% tačnost klasifikacije (Eksperiment 2).

4.8.2 Praktična cijena napada na realni RSA

Eksperiment koristi 64-bitni privatni eksponent radi brzog izvođenja, ali mehanizam je identičan za realni RSA s 2048-bitnim ključem jer se svaki bit napada nezavisno istom tehnikom. Napad se konceptualno skalira linearno jer se informacije o bitovima privatnog eksponenta

²Za Flush+Reload, zaštita je teoretski izvedena: constant-time implementacija izvršava obje operacije uvijek, pa bi ciljna funkcija bila prisutna u cache-u neovisno o vrijednosti bita. Ovo nije eksperimentalno verifikirano u okviru ovog rada, i zahtijeva da obje putanje pristupaju istim cache linijama. Eksperimentalnu verifikaciju ove tvrdnje treba smatrati otvorenim pravcem za buduće istraživanje; u kontekstu ovog rada, zaštitu constant-time implementacije od Flush+Reload napada treba tretirati kao teorijsku pretpostavku, ne empirijsko tvrđenje.

izdvajaju iterativno korištenjem iste metodologije mjerenja: za potpunu rekonstrukciju 2048-bitnog ključa potrebno je $2048 \times 5000 \times 2 \approx 20,5$ miliona mjerenja (5000 mjerenja po bitu, dva mjerenja po paru).

Pri prosječnom trajanju jedne 2048-bitne modularne eksponencijacije od $\approx 1\text{--}5$ ms (realistično za softversku aritmetiku velikih brojeva na standardnom savremenom procesoru), ukupno vrijeme napada iznosi od $\approx 5,7$ sati do $\approx 28,5$ sati. Sa CRT (*Chinese Remainder Theorem* – Kineski teorem o ostacima) optimizacijom koja ubrzava RSA dekriptovanje za faktor $\approx 4\times$, procjena se spušta na $\approx 1,4\text{--}7$ sati. Ovo ilustruje da timing napad nije isključivo teorijski, već je praktično izvodljiv u razumnom vremenskom okviru, pogotovo s obzirom na to da se napad može paralelizovati za pozicije bita.

U mrežnom scenariju svako mjerenje uvodi $\approx 0,1\text{--}1$ ms dodatne latencije, pa bi 20 miliona mjerenja trajalo od nekoliko sati do nekoliko dana, zavisno od mrežne udaljenosti i metode pojačavanja signala [4].

4.8.3 Primjenjivost na moderne RSA biblioteke

Napad demonstriran u ovom radu zasniva se na klasičnoj square-and-multiply implementaciji modularne eksponencijacije koja koristi eksplicitno grananje zavisno od vrijednosti bitova privatnog eksponenta. Savremene kriptografske biblioteke, poput *OpenSSL*-a i *libgcrypt*-a, uglavnom implementiraju različite zaštitne mehanizme kako bi se smanjilo curenje vremenskih informacija, čime se značajno otežava izvođenje klasičnih timing napada.

Ipak, zaštita od timing napada ne zavisi isključivo od algoritma, već i od načina implementacije aritmetike velikih brojeva. Implementacije koje ne koriste operacije konstantnog vremena nad operandima i dalje mogu otkrivati informacije kroz suptilne razlike u vremenu izvršavanja. Zbog toga manje poznate, prilagođene ili nestandardne kriptografske biblioteke mogu ostati ranjive čak i kada koriste moderne algoritamske optimizacije.

Poglavlje 5

Zaključak

5.1 Pregled postignutih rezultata

U ovom radu provedena je eksperimentalna analiza timing i cache timing side-channel napada nad RSA implementacijom zasnovanom na algoritmu kvadriraj-i-množi. Eksperimenti su provedeni na modernom hardveru (Intel i5-1335U, Raptor Lake) pod Ubuntu Linux operativnim sistemom, s preciznim mjerenjem CPU ciklusa. Ukupno je provedeno sedam eksperimenata koji pokrivaju detekciju vremenskog curenja, praktični napad rekonstrukcije ključa, sistematsku analizu svih bit-pozicija, evaluaciju kontramjera, te Flush+Reload cache napad.

Provedena analiza potvrdila je sve tri postavljene hipoteze: vremensko curenje u naivnoj kvadriraj-i-množi implementaciji je mjerljivo i iskoristivo, klasične kontramjere pokazuju ograničenja, a cache bazirani napadi nude znatno jači signal. Zajedno, ovi nalazi eksperimentalno demonstriraju na modernom hardveru ono što teorijska literatura predviđa, da kriptografska korektnost implementacije ne implicira njenu sigurnost.

Eksperimenti su potvrdili postojanje mjerljivog vremenskog curenja od +75,90 CPU ciklusa u naivnoj implementaciji (Cohen-ov $d = 0,134$, $p < 10^{-50}$), koje omogućava potpunu slijepu rekonstrukciju 64-bitnog privatnog ključa sa 100% tačnošću, pri čemu napadački kod nikada nije imao pristup vrijednosti tajnog ključa. Analiza pozicija bita potvrdila je da curenje nije ograničeno na jednu poziciju, već je prisutno na svim 64 pozicijama. Implementacija s konstantnim vremenom potpuno eliminiše curenje ($d \approx 0$, $p = 0,84$), a tačnost napada pada na razinu slučajnog pogađanja ($\approx 50\%$).

5.2 Implikacije za praktičnu kriptografsku sigurnost

5.2.1 Neintuitivnost ranjivosti

Naivna kvadriraj-i-množi implementacija matematički je potpuno ispravna i ranjivost ne leži u matematici nego u fizičkoj realizaciji, što ilustrira ključnu pouku: *kriptografska sigurnost zahtijeva razmatranje izvedbe, a ne samo matematičke ispravnosti*. Ova neintuitivnost čini side-channel ranjivosti posebno opasnim: ne može ih otkriti ni formalna provjera ispravnosti ni standardno testiranje.

5.2.2 Zavisnost od kompajlera nije sigurnost

Eksperiment 5 potvrđuje da oslanjanje na kompajlerske optimizacije za sigurnosne mjere nije prihvatljiva opcija: nijedan testirani nivo nije generisao kod s konstantnim vremenom. Programer koji se osloni na kompajler ostaje nesvjesno izložen.

5.2.3 Blinding ima ograničenja

Eksperiment 6 pokazuje da blinding, iako efikasna kontramjera za operand-zavisne timing leake u realnom RSA [4], ne štiti od vremenskog curenja koje zavisi od grananja. Sigurna implementacija zahtijeva kombinaciju više tehnika: vremenski konstantne algoritme, blinding za maskiranje operand-zavisnih operacija u višepreciznoj aritmetici, te vremenski konstantne aritmetičke primitive.

5.3 Ograničenja i mogući pravci daljeg istraživanja

5.3.1 Ograničenja

1. **Veličina ključa (64 vs 2048+ bita).** Eksperiment koristi 64-bitni privatni eksponent i 61-bitni Mersenne modulus. Stvarni RSA koristi eksponente od 2048 do 4096 bita. Mehanizam je identičan za svaki bit, i napad se sa istim metodologijama skalira linearno s brojem bita, ali ukupno trajanje eksperimenta i broj potrebnih mjerenja proporcionalno rastu. Zaključci o detektabilnosti signala (Cohen-ov d , statistički testovi) direktno se prenose na veće ključeve jer su zasnovani na analizi po bitu.
2. **Jedna platforma (x86_64, Intel Raptor Lake).** Svi eksperimenti provedeni su na jednoj mašini s jednim procesorom. Vrijednosti timing signala (≈ 76 ciklusa), razina šuma ($\sigma \approx 567$) i potreban broj mjerenja ($K = 2000$) specifični su za ovu arhitekturu, frekvenciju i njene karakteristike. ARM procesori, starije generacije Intela ili AMD platforme mogu imati drugačije karakteristike, što bi utjecalo na konkretne vrijednosti, ali ne i na korišten mehanizam curenja informacija.
3. **Lokalni model prijetnji (*threat model*).** Napadač u ovom eksperimentu pokreće kod na istoj mašini kao i žrtva, bez mrežnog kašnjenja. Brumley i Boneh [4] demonstrirali su da je napad izvodljiv i u mrežnom okruženju, ali uz daleko veći broj mjerenja i komplikovanije metode pojačavanja signala. Naš lokalni eksperimentalni postav idealizira uslove i daje značajno čišći signal od realnog mrežnog napada.
4. **Algoritam kvadriraj-i-množi ili moderni algoritmi.** Moderne kriptografske biblioteke (OpenSSL, libcrypto) rijetko koriste naivni kvadriraj-i-množi s eksplicitnim grananjem. Ovaj rad demonstrira glavni princip, ali direktna primjena na moderne biblioteke zahtijevala bi analizu specifičnih implementacija. Ipak, razumijevanje naivne implementacije nužan je uslov za analizu komplikovanijih varijanti.

5.3.2 Pravci daljeg istraživanja

Prirodni nastavak ovog rada mogao bi ići u dva smjera, i to prema realističnijem modelu prijetnji, ili prema složenijim kriptografskim primitivama:

- Analizu napada u mrežnom scenariju, replikacijom eksperimenta na mreži.

- Evaluaciju Prime+Probe napada koji ne zahtijeva dijeljenu memoriju i radi između virtualnih mašina.
- Analizu timing napada na eliptičnu kriptografiju (ECDSA, ECDH), gdje su side-channel ranjivosti kompleksnije ali prisutne.
- Analizu timing napada nad postkvantnim algoritmima.

Rezultati potvrđuju da timing i cache side-channel napadi na RSA imaju praktičnu primjenjivost na modernom hardveru, i dok kriptografski algoritmi postaju sve robusniji, sigurnost implementacije ostaje otvoren problem koji zahtijeva jednaku pažnju kao i matematički temelji na kojima počivaju.

Prilozi

Prilog A

Konfiguracija hardverskog okruženja

Ovaj prilog dokumentuje detaljne korake konfiguracije hardverskog okruženja primijenjene prije pokretanja eksperimenata. Ove mjere osiguravaju reproducibilnost i minimizaciju sistematskog šuma.

A.1 Isključivanje Turbo Boost-a

Intel Turbo Boost dinamički podiže frekvenciju procesora (do 4,6 GHz na i5-1335U) ovisno o termičkom stanju. Budući da TSC raste po konstantnoj stopi (nominalnoj frekvenciji), aktivan Turbo znači da isti broj TSC ciklusa odgovara različitoj količini stvarnog rada. Isključivanje:

Program A.1: Isključivanje Intel Turbo Boost-a

```
echo 1 | sudo tee /sys/devices/system/cpu/intel_pstate/no_turbo
```

A.2 Performance CPU governor

Podrazumijevani powersave governor (korišten s intel_pstate driverom na ovoj platformi) dinamički prilagođava frekvenciju, što bi uzrokovalo da početne iteracije rade na nižoj frekvenciji:

Program A.2: Postavljanje performance governor-a

```
sudo cpupower frequency-set -g performance
```

A.3 CPU pinning

Intel i5-1335U ima P-core (Raptor Cove, out-of-order, 3,4 GHz baza) i E-core (Gracemont, out-of-order, 2,5 GHz) jezgre s različitim latencijama. Bez eksplicitnog odabira, Linux scheduler može migrirati procese između jezgri:

Program A.3: Pokretanje eksperimenta s fiksiranjem na P-core 0

```
taskset -c 0 ./experiment
```

Dodatno, eksperimentalni program programski poziva sched_setaffinity() kao drugi nivo osiguranja.

A.4 Zagrijavanje cache-a

Prvih 10 000 iteracija eksperimenta je faza zagrijavanja čija se mjerenja odbacuju. Svrha:

- Populacija L1/L2 cache memorije relevantnim kodom i podacima
- Stabilizacija branch prediktora
- Termalna stabilizacija procesora (eliminacija thermal throttling-a)

A.5 Verifikacija TSC frekvencije

Program mjeri TSC frekvenciju poređenjem TSC-a i `CLOCK_MONOTONIC` sata tokom 100 ms intervala. Izmjereno: $\approx 2,101$ GHz (odgovara nominalnoj frekvenciji).

A.6 Provjera assembly izlaza

Za verifikaciju da kompajler ne generiše `CMOV` instrukcije:

Program A.4: Provjera assembly koda za branch instrukcije

```
make check-asm-all
# Ocekivani izlaz: jne/je (branch), nikad cmov
```

A.7 Assembly izlaz: implementacija sa grananjem i bez grananja

Za verifikaciju da je timing ranjivost prisutna na nivou mašinskog koda, ispod su prikazani ključni dijelovi assembly izlaza dobijenog naredbom `objdump -d`. Korišten je GCC 13.3 na Ubuntu 24.04.

A.7.1 Ranjiva implementacija (eksplicitni branch)

Isječak koda A.5 prikazuje kritični uslovni skok u funkciji `mod_exp_vulnerable`. Instrukcija `je` (`jump if equal / jump if zero`) preskače blok množenja kada je bit eksponenta 0, čime nastaje mjerljiva timing razlika.

Program A.5: Ključni uslovni skok (*branch*) u `mod_exp_vulnerable` – stvarni `objdump` izlaz (GCC 13.3, `-O0`, Intel sintaksa)

```
; Testiranje bita eksponenta: if ((exp >> i) & 1)
208b:    mov     eax,DWORD PTR [rbp-0xc]        ; učitaj i (loop counter
)
208e:    mov     rdx,QWORD PTR [rbp-0x20]       ; učitaj exp
2092:    shr     rax,rdx,rax                    ; rax = exp >> i
2097:    and     eax,0x1                        ; rax = (exp >> i) & 1
209a:    test    rax,rax
209d:    je      20ba <mod_exp_vulnerable+0x89> ; USLOVNI SKOK:
      preskok za bit=0
; --- blok mnozenja: izvrsava se SAMO za bit=1 ---
```

```
209f:    mov     rdx,QWORD PTR [rbp-0x28]    ; učitaj mod
20a3:    mov     rcx,QWORD PTR [rbp-0x18]    ; učitaj base
20a7:    mov     rax,QWORD PTR [rbp-0x8]     ; učitaj result
20ab:    mov     rsi,rcx
20ae:    mov     rdi,rax
20b1:    call    1f8d <mulmod>                ; poziv mulmod(result,
    base, mod)
20b6:    mov     QWORD PTR [rbp-0x8],rax      ; pohrani result
; --- obje grane spajaju se ovdje ---
20ba:    sub     DWORD PTR [rbp-0xc],0x1     ; i--
```

A.7.2 Konstantna vremenska implementacija (branchless)

Za usporedbu, isječak koda A.6 prikazuje `ct_select()` selekciju u konstantnoj vremenskoj implementaciji. Umjesto uslovnog skoka, koriste se isključivo aritmetičke i logičke instrukcije koje se izvršavaju u konstantnom vremenu bez obzira na vrijednost bita.

Program A.6: Selekcija bez grananja (*branchless*) u `mod_exp_constant_time`

```
; ct_select(bit, mul_result, result) -- bez grananja
movq   %rdi, %rax          ; load bit
andl   $0x1, %eax          ; bit & 1
negq   %rax                ; mask = -(bit & 1)
movq   %rax, %rcx          ; rcx = mask
andq   %rsi, %rcx          ; mul_result & mask
notq   %rax                ; ~mask
andq   %rdx, %rax          ; result & ~mask
orq    %rcx, %rax          ; combine: finalni rezultat
```

CPU izvršava identičnu sekvencu instrukcija bez obzira na vrijednost bita i nema uslovnih skokova, nema branch predikcije, nema timing razlike.

Literatura

- [1] Rivest, R. L., Shamir, A., Adleman, L., “A method for obtaining digital signatures and public-key cryptosystems”, Communications of the ACM, Vol. 21, No. 2, 1978, str. 120–126.
- [2] Kocher, P. C., “Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems”, in Advances in Cryptology – CRYPTO 1996, ser. Lecture Notes in Computer Science, Vol. 1109. Springer, 1996, str. 104–113.
- [3] Dhem, J.-F., Koeune, F., Leroux, P.-A., Mestré, P., Quisquater, J.-J., Willems, J.-L., “A practical implementation of the timing attack”, in Smart Card Research and Advanced Applications – CARDIS 1998, ser. Lecture Notes in Computer Science, Vol. 1820. Springer, 1998, str. 167–182, zbornik objavljen 2000.
- [4] Brumley, D., Boneh, D., “Remote timing attacks are practical”, in Proceedings of the 12th USENIX Security Symposium. USENIX Association, 2003, str. 1–14.
- [5] Ge, Q., Yarom, Y., Cock, D., Heiser, G., “A survey of microarchitectural timing attacks and countermeasures on contemporary hardware”, Journal of Cryptographic Engineering, Vol. 8, No. 1, 2018, str. 1–27.
- [6] Cocks, C. C., “A note on “Non-Secret Encryption””, Communications-Electronics Security Group (CESG), GCHQ, Tech. Rep., 1973, declassified and published 1997.
- [7] Lenstra, A. K., Lenstra, H. W., (ur.), The Development of the Number Field Sieve, ser. Lecture Notes in Mathematics. Springer, 1993, Vol. 1554.
- [8] Shor, P. W., “Algorithms for quantum computation: Discrete logarithms and factoring”, in Proceedings of the 35th Annual Symposium on Foundations of Computer Science. IEEE, 1994, str. 124–134.
- [9] National Institute of Standards and Technology, “Post-quantum cryptography standards: FIPS 203, 204, 205”, U.S. Department of Commerce, National Institute of Standards and Technology, Tech. Rep., 2024, module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203), Module-Lattice-Based Digital Signature Standard (FIPS 204), Stateless Hash-Based Digital Signature Standard (FIPS 205).
- [10] Menezes, A. J., van Oorschot, P. C., Vanstone, S. A., Handbook of Applied Cryptography. CRC Press, 1996, dostupno na: <https://cacr.uwaterloo.ca/hac/>.
- [11] Mangard, S., Oswald, E., Popp, T., Power Analysis Attacks: Revealing the Secrets of Smart Cards. Springer, 2007.

- [12] Kocher, P., Jaffe, J., Jun, B., “Differential power analysis”, in Advances in Cryptology – CRYPTO 1999, ser. Lecture Notes in Computer Science, Vol. 1666. Springer, 1999, str. 388–397.
- [13] Gandolfi, K., Mourtel, C., Olivier, F., “Electromagnetic analysis: Concrete results”, in Cryptographic Hardware and Embedded Systems – CHES 2001, ser. Lecture Notes in Computer Science, Vol. 2162. Springer, 2001, str. 251–261.
- [14] Yarom, Y., Falkner, K., “FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack”, in Proceedings of the 23rd USENIX Security Symposium. USENIX Association, 2014, str. 719–732.
- [15] Genkin, D., Shamir, A., Tromer, E., “RSA key extraction via low-bandwidth acoustic cryptanalysis”, in Advances in Cryptology – CRYPTO 2014, ser. Lecture Notes in Computer Science, Vol. 8616. Springer, 2014, str. 444–461.
- [16] Welch, B. L., “The generalization of ‘Student’s’ problem when several different population variances are involved”, Biometrika, Vol. 34, No. 1–2, 1947, str. 28–35.
- [17] Mann, H. B., Whitney, D. R., “On a test of whether one of two random variables is stochastically larger than the other”, The Annals of Mathematical Statistics, Vol. 18, No. 1, 1947, str. 50–60.
- [18] Cohen, J., Statistical Power Analysis for the Behavioral Sciences, 2nd ed. Lawrence Erlbaum Associates, 1988.
- [19] Efron, B., “Bootstrap methods: Another look at the jackknife”, The Annals of Statistics, Vol. 7, No. 1, 1979, str. 1–26.
- [20] Intel Corporation, Intel 64 and IA-32 Architectures Software Developer’s Manual, Intel Corporation, 2023, volume 3B, Chapter 17: Time-Stamp Counter.
- [21] Paoloni, G., “How to benchmark code execution times on Intel IA-32 and IA-64 instruction set architectures”, Intel Corporation, Tech. Rep. 324264-001, 2010.
- [22] Intel Corporation. (2022) 13th generation intel core processor datasheet, dostupno na: <https://www.intel.com/content/www/us/en/products/docs/processors/core/13th-gen-core-datasheet.html>
- [23] Wilcox, R. R., Introduction to Robust Estimation and Hypothesis Testing, 2nd ed. Academic Press, 2005.